

Apostila Final — Teoria da Informação

Edição revisada e ampliada — com exemplos aprofundados, figuras e leituras complementares

Sumário

Apresentação desta edição	5
PARTE I — FUNDAMENTOS	6
Capítulo 1 — Dados, Informação e Conhecimento	7
1.1 Por que começar por aqui?	7
1.2 A cadeia Dado → Valor → Informação → Conhecimento	7
1.3 A Era da Informação: de “poucos dados” a “excesso de dados”	8
1.4 Tipos de dados: estruturados, semi-estruturados e não estruturados	9
1.5 Vocabulário: atributos e objetos	10
Classificação dos atributos	10
Formas de representação de dados	10
1.6 O pipeline completo: da coleta ao conhecimento	11
Capítulo 2 — Fundamentos de Probabilidade	13
2.1 Axiomas de probabilidade (Kolmogorov)	13
2.2 Probabilidade condicional e independência	13
2.3 Função de distribuição, densidade e variáveis discretas/contínuas	14
2.4 Valor esperado, momentos, variância	15
2.5 Covariância e correlação	16
2.6 Mudança de variáveis (Jacobiano)	16
2.7 Três desigualdades fundamentais	16
Desigualdade de Jensen	17
Desigualdade de Markov	17
Desigualdade de Chebyshev	18
Capítulo 3 — Estatística Descritiva e Pré-processamento de Dados	19
3.1 Medidas de posição	19
3.2 Medidas de dispersão	20
3.3 Distribuição e frequência	20
3.4 Quartis e percentis	21
3.5 Correlação amostral	21
3.6 Recomendações de uso de métricas	21
3.7 Transformação de dados	22
3.8 Normalização de dados	22
Método 1 — Norma constante (normalização local)	22
Método 2 — Mudança de escala (min-max, normalização global)	23

Método 3 — Padronização (z-score)	23
Normalização como transformação linear e suas propriedades	24
Capítulo 4 — Distribuições de Probabilidade Contínuas	25
4.1 Distribuição Exponencial e a propriedade de falta de memória	26
4.2 Distribuição Gama e sua relação com Exponencial, Erlang e Qui-quadrado	27
4.3 Relação Gama-Beta	27
4.4 Weibull como distribuição “camaleão”	27
4.5 Distribuição Normal e o Teorema do Limite Central	28
Capítulo 5 — Inferência, Processos Estocásticos e Técnicas de Predição	29
5.1 Teorema de Bayes	29
5.2 Processos estocásticos e Cadeias de Markov	30
5.3 Teoria da decisão	31
5.4 Técnicas de predição (visão panorâmica)	31
PARTE II — TEORIA DA INFORMAÇÃO CLÁSSICA (SHANNON)	33
Capítulo 6 — Quantidade de Informação e Entropia	34
6.1 Autoinformação (informação de um evento)	34
6.2 Entropia de Shannon	36
Caso binário	36
6.3 Extensão de uma fonte discreta sem memória	37
Capítulo 7 — Entropia Conjunta, Condicional, Divergência KL e Informação Mútua	39
7.1 Entropia conjunta	39
7.2 Entropia condicional	39
7.3 Regra da cadeia	40
7.4 Divergência de Kullback-Leibler (entropia relativa / entropia cruzada) . .	40
7.5 Informação Mútua	41
7.6 Ganho de Informação e árvores de decisão	42
Capítulo 8 — Entropia Diferencial e Generalizações da Entropia	44
8.1 Entropia diferencial	44
8.2 Propriedades gerais (válidas nos dois casos)	45
8.3 Outras definições de entropia	45
Entropia de Rényi	45
Entropia de Boltzmann-Gibbs (origem física do nome “entropia”)	46
Entropia de Tsallis	46
Entropia de von Neumann (domínio quântico)	46
Entropia Espectral	47
Capítulo 9 — Codificação de Fonte e Compressão de Dados	48
9.1 O problema da codificação de fonte	48
9.2 O Primeiro Teorema de Shannon (Teorema da Codificação de Fonte) . . .	48
9.3 Códigos prefixo e a Inequação de Kraft-McMillan	49
9.4 Codificação de Huffman	49
Algoritmo (passo a passo)	50

Exemplo resolvido	50
Não-unicidade e limitações	51
9.5 Codificação Lempel-Ziv-Welch (LZW)	52
Algoritmo de compressão	52
Exemplo resolvido passo a passo	52
9.6 Compressão com e sem perdas — visão geral	53
9.7 Estimando entropia a partir de dados reais	53
Capítulo 10 — Capacidade de Canal, Ruído e o Limite de Shannon	55
10.1 O problema da codificação de canal	55
10.2 Segundo Teorema de Shannon (Teorema da Codificação do Canal)	55
10.3 Capacidade de canal: definição geral	56
Canal Binário Simétrico (BSC)	56
Canal com Apagamento (BEC) e canal combinado	57
10.4 Terceiro Teorema de Shannon: capacidade do canal gaussiano (Shannon-Hartley)	58
Leitura da fórmula	58
Exemplo numérico resolvido	59
Eficiência espectral e o limite de Shannon em E_b/N_0	59
Capítulo 11 — Códigos Corretores de Erro	61
11.1 Motivação	61
11.2 Categorias de códigos	61
11.3 Técnicas de detecção e correção	62
Exemplo resolvido completo — Hamming(7,4)	62
11.4 Taxa de código e o trade-off fundamental	62
11.5 Aplicações modernas	63
Capítulo 12 — Complexidade de Kolmogorov	64
12.1 Uma segunda forma de medir informação	64
12.2 Definição formal	65
Teorema da Invariância (existência de função ótima)	65
12.3 Strings incompressíveis	65
12.4 O Teorema de Gödel via Complexidade de Kolmogorov	66
12.5 Aleatoriedade formalizada sem probabilidade	67
12.6 Complexidade de Kolmogorov como precursor da entropia	67
PARTE III — BIG DATA, APRENDIZADO DE MÁQUINA E RECONHECIMENTO DE PADRÕES	69
Capítulo 13 — Big Data: Frameworks e Processamento Distribuído	70
13.1 Por que processamento distribuído?	70
13.2 MapReduce	70
Exemplo canônico: WordCount	70
Operações da álgebra relacional via MapReduce	71
Custo computacional	71
13.3 Processamento em fluxo (Streaming) e Apache Spark	72
13.4 Convergência com IA	72

Capítulo 14 — Fundamentos de Redes Neurais e Aprendizado	74
14.1 Por que aprendizado de máquina?	74
14.2 Métricas de avaliação	74
14.3 O mecanismo de treinamento	75
Erro e função de perda	75
Gradiente descendente	75
Variantes do Gradiente Descendente	76
Funções de ativação (a não-linearidade)	77
Backpropagation (regra da cadeia)	77
14.4 Overfitting e boas práticas	78
Capítulo 15 — Análise de Componentes Principais (PCA)	79
15.1 Contexto histórico	79
15.2 O problema: redução de dimensionalidade preservando informação	79
15.3 O algoritmo PCA, passo a passo	79
15.4 Por que isso funciona? A diagonalização da covariância	80
15.5 Interpretação geométrica: mudança de base	81
15.6 Aplicações	82
Seleção de atributos	82
Redução de dimensionalidade	82
Variância total e variância explicada	82
Capítulo 16 — Síntese Final e Mapa Conceitual	84
16.1 O fio condutor do curso	84
16.2 Mapa de conexões entre os capítulos	84
16.3 Tabela-resumo das grandezas fundamentais	85
16.4 Dicas de estudo e revisão	85
Referências e leituras complementares	87

Apresentação desta edição

Esta é uma versão **aprimorada** da apostila consolidada da disciplina de Teoria da Informação (material do Prof. Dr. Julio César Santos dos Anjos, apostila de estatística da UFC, material de PCA e o capítulo de Complexidade de Kolmogorov de Durand & Zvonkin). O escopo dos 16 capítulos originais foi mantido integralmente; o que mudou:

- **Explicações aprofundadas:** cada conceito central ganhou parágrafos adicionais de intuição, motivação histórica e conexão com os demais capítulos.
- **Mais exemplos resolvidos:** novos exercícios numéricos, passo a passo, foram adicionados em quase todos os capítulos (Bayes com teste diagnóstico, entropia de fontes reais, canal conjunto completo, Hamming(7,4), LZW detalhado, PCA com autovetores explícitos, entre outros).
- **Figuras:** catorze figuras foram produzidas especificamente para este material — curvas de entropia, desigualdade de Jensen, árvore de Huffman, capacidade do BSC, limite de Shannon, PCA, diagnóstico de overfitting etc.
- **Novas fontes:** cada capítulo indica, ao final, leituras complementares específicas (livros-texto clássicos, tutoriais e artigos), além da bibliografia geral consolidada ao fim.
- As **caixas de nota** da versão anterior (correções de erros dos slides originais) foram preservadas.

Como usar: os capítulos seguem uma progressão lógica (fundamentos → teoria clássica de Shannon → aplicações em Big Data/ML), mas cada capítulo permanece relativamente autocontido, podendo ser consultado isoladamente para revisão. Fórmulas estão em LaTeX.

PARTE I — FUNDAMENTOS

Capítulo 1 — Dados, Informação e Conhecimento

1.1 Por que começar por aqui?

Antes de qualquer formalismo matemático, a disciplina começa com uma pergunta aparentemente simples: **o que é informação?** Essa pergunta tem uma longa história — desde os primeiros registros humanos (como as pinturas rupestres do Parque da Pedra Pintada, na Amazônia) até os bilhões de gigabytes gerados por segundo hoje em dia, a humanidade sempre lidou com o problema de registrar, transmitir e interpretar dados.

Há, na verdade, **duas tradições** para responder a essa pergunta, e o curso percorre ambas:

1. A tradição **semântica/organizacional** (este capítulo): informação é dado interpretado em contexto. É a visão da Ciência da Informação e da Ciência de Dados.
2. A tradição **matemática/quantitativa** (Parte II): informação é redução de incerteza, medida em bits, independentemente do *significado* da mensagem. Shannon foi explícito sobre isso em 1948 — sua teoria trata da *forma* e da *quantidade*, não do conteúdo semântico. As duas visões não competem: a primeira diz *o que fazer* com dados; a segunda diz *quais são os limites físicos e matemáticos* do que é possível fazer (comprimir, transmitir, inferir).

A ideia central que percorre todo o curso é que **dado bruto não é a mesma coisa que informação útil**. Um número, uma imagem ou um sinal, isolados, não “significam” nada por si só — é o processamento, o contexto e a interpretação que transformam dado em informação, e informação em conhecimento.

1.2 A cadeia Dado → Valor → Informação → Conhecimento

Essa progressão (uma variação da conhecida hierarquia **DIKW** — *Data, Information, Knowledge, Wisdom* — cuja formulação moderna é geralmente atribuída a Russell Ackoff, 1989, e revisada criticamente por Rowley, 2007) pode ser resumida assim:

- **Dado**: um registro bruto que, por si só, não carrega significado. Exemplo: o número 37.

- **Valor:** propriedade que se agrega ao dado para lhe dar relevância dentro de um contexto. Exemplo: 37 é uma *temperatura*.
- **Informação:** é o dado ao qual se atribuiu significado — um valor interpretado dentro de um contexto. Exemplo: “a temperatura corporal do paciente às 8h era 37 °C”.
- **Conhecimento:** resultado do cruzamento de diferentes informações, do qual se pode inferir e tomar decisões, gerando por sua vez nova informação (um ciclo de retroalimentação). Exemplo: cruzando a temperatura de 37 °C com o histórico do paciente e outros sintomas, um médico conclui que há um quadro febril e decide um tratamento — essa decisão gera novos dados/informações que realimentam o ciclo.

Esse ciclo (dado → conhecimento → nova informação) **não é linear, é iterativo**: o conhecimento gerado hoje se torna insumo (dado) para análises futuras.

Exemplo adicional (IoT/agricultura de precisão): um sensor de umidade do solo transmite a leitura 0.18. Isso é um *dado*. Sabendo que a escala é fração volumétrica de água, 0.18 ganha *valor*. Combinado com a localização do talhão e a hora, vira *informação*: “o setor B3 está com 18% de umidade às 14h”. Cruzando isso com a curva de retenção daquele tipo de solo e a previsão do tempo, o agrônomo (ou um modelo preditivo — Parte III) produz *conhecimento*: “irrigar o setor B3 por 40 minutos hoje evita estresse hídrico”. A decisão gera novos dados de consumo de água — e o ciclo recomeça.

1.3 A Era da Informação: de “poucos dados” a “excesso de dados”

Historicamente, a preocupação central da humanidade em relação à informação mudou de natureza:

Época	Problema central	Marco tecnológico
Passado remoto	Falta de informação — como transmiti-la?	Tradição oral
Invenção da escrita	Como representá-la de forma durável?	Escrita, depois imprensa
Era digital / Big Data	Excesso de informação — como processá-la?	Computadores, internet, sensores

Hoje vivemos o polo oposto do problema original: não falta dado, **sobra**. Alguns números que ilustram essa explosão (citados no material do curso, com fonte statista.com e discovery.com):

- O volume mundial de dados gerados é medido em **zettabytes** (1 ZB = 10^9 TB).
- Há 6,65 bilhões de usuários de smartphone no mundo — mais de 83% da população — e 60% do tráfego de internet vem de dispositivos móveis.
- Cada olho humano capta cerca de 576 megapixels por cena — quase 9 vezes mais que os 63 MP de uma câmera de iPhone 15 Pro. Essa comparação é

usada para argumentar que **lidar com grandes volumes de dados é, em certo sentido, “natural”**: o próprio cérebro humano constantemente filtra e comprime informação sensorial bruta em percepções úteis. (Curiosamente, essa é uma antecipação intuitiva do que a Parte II formaliza: perceber é *comprimir com perdas* — o nervo óptico transmite uma fração mínima do que a retina capta, priorizando bordas, movimento e contraste.)

A metáfora do iceberg: os dados que efetivamente coletamos e vemos são apenas a ponta visível de um iceberg — a maior parte do “valor” informacional está latente e só emerge depois de processamento, análise e modelagem. Essa é a motivação central para todo o restante do curso: aprender técnicas (estatísticas, de teoria da informação e de aprendizado de máquina) capazes de extrair esse valor oculto.

1.4 Tipos de dados: estruturados, semi-estruturados e não estruturados

Uma classificação central em Ciência de Dados, usada ao longo de todo o curso:

(a) Dados estruturados — Organização rígida e bem definida, tipicamente em tabelas (linhas e colunas). Fácil de consultar e processar (a linguagem SQL existe justamente para isso), mas caro de reestruturar: alterar o esquema de um banco em produção é uma operação delicada. Exemplos: bancos de dados relacionais (SQL), planilhas.

(b) Dados semi-estruturados — Não seguem um modelo tabular fixo, mas têm **metadados** (tags, marcadores) que permitem alguma organização. São o formato dominante na comunicação entre sistemas na web (APIs REST trocam JSON; configurações usam XML/YAML). Exemplos clássicos: XML e JSON. Compare a mesma informação nos dois formatos:

```
<user>
  <id>11</id>
  <animal>Bob</animal>
  <idade>2</idade>
</user>

{ "user": { "id": "11", "animal": "Bob", "idade": "2" } }
```

Note que o JSON expressa a mesma estrutura com menos caracteres — um exemplo cotidiano de que a *mesma informação* admite representações de tamanhos diferentes, tema que o Capítulo 9 (compressão) e o Capítulo 12 (Complexidade de Kolmogorov) tratarão com rigor matemático.

(c) Dados não estruturados — Sem padrão definido, não organizáveis em tabelas diretamente. Cerca de **80% dos dados produzidos no mundo são não estruturados** (estatística amplamente citada na literatura de Big Data). Exemplos: e-mails, textos livres, páginas web, imagens, áudio, vídeo — normalmente exigem uma etapa de **pré-processamento** antes de qualquer análise. Boa parte da revolução recente do *deep learning* (Capítulo 14) vem justamente da capacidade de extrair *features* automaticamente desse tipo de dado.

1.5 Vocabulário: atributos e objetos

Um **atributo** é uma propriedade ou característica de um objeto (também chamado de *variável*, *campo*, *característica* ou *feature* — termos usados de forma intercambiável em diferentes comunidades: estatística, bancos de dados, aprendizado de máquina). Exemplos: cor dos olhos, temperatura.

Um **objeto** é uma coleção de atributos (também chamado de *registro*, *ponto*, *caso*, *amostra*, *entidade* ou *instância*). Quando todos os objetos compartilham o mesmo conjunto de atributos numéricos, podem ser vistos como **pontos em um espaço multidimensional** — cada atributo é uma dimensão — e organizados em uma matriz $m \times n$ (m objetos/linhas, n atributos/colunas). Essa é exatamente a representação usada mais adiante em PCA (Capítulo 15) e em quase todo algoritmo de aprendizado de máquina.

Classificação dos atributos

Categoria	Subtipo	Característica	Exemplos
Categórico	Nominal	Sem ordem	Cor dos olhos, forma de pagamento
	Ordinal	Ordenável, mas não comparável numericamente	Ranking (bom/médio/ruim), faixa etária
Numérico	Discreto	Contagem	Número de filhos, idade em anos
Binário	Contínuo	Escala contínua	Temperatura, peso
	Caso especial	Sim/não, existe/não existe	—

Essa taxonomia (equivalente às clássicas **escalas de medição de Stevens**, 1946: nominal, ordinal, intervalar, razão) determina, mais adiante, quais métricas estatísticas e quais técnicas de normalização fazem sentido para cada variável (ver Capítulo 3). Um erro conceitual comum — e frequente na prática — é calcular média de um atributo *ordinal* (ex.: média de “bom/médio/ruim” codificado como 3/2/1): o resultado é numérico, mas a operação pressupõe distâncias iguais entre as categorias, o que a escala ordinal não garante.

Formas de representação de dados

- **Documentos (bag-of-words)**: cada documento vira um vetor cujo i -ésimo componente é o número de ocorrências do i -ésimo termo do vocabulário — perde a ordem das palavras, mas é útil para muitas tarefas de PLN (Processamento de Linguagem Natural). Refinamentos comuns: ponderação **TF-IDF** (penaliza termos frequentes em todos os documentos) e, mais recentemente, *embeddings* densos.

- **Transações:** cada registro é um conjunto de itens, representável como vetor binário de presença/ausência — base da análise de cesta de compras (*market basket analysis*).
- **Dados ordenados:** sequências (ex.: genoma, séries temporais).
- **Grafos:** nós e arestas (ex.: links entre páginas web, redes sociais, redes de citação).

1.6 O pipeline completo: da coleta ao conhecimento

O curso apresenta um pipeline de cinco etapas para transformar dado bruto em conhecimento (uma versão prática do processo clássico de **KDD** — *Knowledge Discovery in Databases*, formalizado por Fayyad, Piatetsky-Shapiro & Smyth, 1996 — embora os slides originais não usem esse nome):

1. **Seleção:** extração dos dados de sua origem (pode ser um processo contínuo/*streaming* em sistemas de Big Data, e frequentemente recursivo, coletando em diferentes níveis/fontes).
2. **Pré-processamento:** limpeza (remoção de ruído/inconsistências), integração (combinação de múltiplas fontes) e seleção/redução dos dados relevantes. Os problemas típicos encontrados aqui são:
 - *Incompletude*: falta de valores, atributos ou objetos inteiros.
 - *Inconsistência*: valores fora do domínio esperado ou muito discrepantes.
 - *Ruído*: variações indesejadas e nem sempre identificáveis (ex.: “chuvisco” em vídeo, “chiado” em áudio). — Note que “ruído” reaparecerá, com definição matemática precisa, no Capítulo 10 (canal ruidoso).
3. **Transformação:** converter dados descritivos em formato quantitativo e padronizado — por exemplo, transformar um horário “8h-12h, 14h-17h” em categorias “manhã/tarde” (ou em códigos “0”/“1”), ou extrair *features* de uma imagem via convolução. Inclui a **normalização**, tratada em detalhe no Capítulo 3.
4. **Extração de conhecimento (mineração):** aplicação de algoritmos de análise descritiva (tendência central, dispersão, visualização), agrupamento (*clustering*), predição (classificação/regressão), associação (quais atributos ocorrem juntos) e detecção de anomalias.
5. **Interpretação:** avaliação dos resultados, buscando conhecimento verdadeiramente útil e não trivial. Se os resultados não forem satisfatórios, volta-se às etapas 1 ou 2 — o pipeline é **cíclico**, não estritamente sequencial, especialmente em aprendizado supervisionado/semi-supervisionado.

Na prática da indústria, estima-se que as etapas 1-3 consumam **60% a 80% do tempo** de um projeto de dados — a mineração propriamente dita (etapa 4) costuma ser a parte menor. Isso justifica o peso que o Capítulo 3 dá ao pré-processamento.

Motivação real (exemplo astronômico): em 2009, o telescópio Hubble ajudou a identificar o exoplaneta GJ1214b (a 40 anos-luz da Terra); somente após um longo processo de análise dos dados brutos de trânsito planetário — dois anos depois — pesquisadores confirmaram, em 2011, que o planeta continha mais água que a Terra. O exemplo ilustra concretamente a tese central deste capítulo: **dado bruto ≠ informação útil; é o processamento que revela o conhecimento.**

Leituras complementares deste capítulo: TAN, STEINBACH & KUMAR, *Introduction to Data Mining* (caps. 1-2, excelente para a taxonomia de atributos); ROWLEY, J. "The wisdom hierarchy: representations of the DIKW hierarchy", *Journal of Information Science*, 2007; FAYYAD et al., "From Data Mining to Knowledge Discovery in Databases", *AI Magazine*, 1996; GLEICK, J. *A Informação: uma história, uma teoria, uma enxurrada*, 2011 (leitura não técnica, panorâmica e histórica).

Capítulo 2 — Fundamentos de Probabilidade

Toda a Teoria da Informação se apoia em probabilidade: entropia é definida a partir de probabilidades de símbolos, capacidade de canal a partir de distribuições de entrada/saída, etc. Este capítulo revisa o ferramental necessário.

2.1 Axiomas de probabilidade (Kolmogorov)

Seja S um espaço amostral (conjunto de todos os resultados possíveis) contido em um espaço de eventos F (uma coleção — tecnicamente uma σ -álgebra — de todos os subconjuntos de S aos quais se pode atribuir uma probabilidade). Para eventos mutuamente exclusivos $A \cdot B = \emptyset$ (ou seja, $A \cap B = \emptyset$):

$$P(A) \geq 0, \quad P(S) = 1, \quad A \cdot B = \emptyset \implies P(A + B) = P(A) + P(B)$$

Esses são os três **axiomas de Kolmogorov** (1933): não-negatividade, normalização (a probabilidade do espaço todo é 1) e aditividade para eventos disjuntos. Toda a teoria de probabilidade moderna é construída a partir desses três postulados.

Vale destacar a força desse resultado: propriedades aparentemente “óbvias” como $P(A^c) = 1 - P(A)$, $P(\emptyset) = 0$, $P(A) \leq 1$ e a regra da união $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ **não são axiomas** — todas se *demonstram* a partir dos três postulados. Exercício rápido: prove $P(A^c) = 1 - P(A)$ usando que A e A^c são disjuntos e $A \cup A^c = S$.

Também é útil conhecer as três *interpretações* usuais de probabilidade, pois todas aparecem no curso: **clássica** (casos favoráveis / casos possíveis, para espaços equiprováveis finitos), **frequentista** (limite da frequência relativa em repetições — é a leitura por trás da estimação de entropia em dados reais, Capítulo 9) e **subjettiva/bayesiana** (grau de crença atualizável por evidência — é a leitura por trás do Teorema de Bayes, Capítulo 5). Os axiomas de Kolmogorov valem para as três.

2.2 Probabilidade condicional e independência

Probabilidade condicional — probabilidade de A dado que B já ocorreu (o espaço amostral é “restrito” a B):

$$P(A|B) \equiv \frac{P(AB)}{P(B)}, \quad P(B) > 0$$

Exemplo resolvido (intuição da restrição do espaço amostral): lança-se um dado honesto. Seja $A = \text{"sair 2"}$ e $B = \text{"sair número par"}$. Sem informação, $P(A) = 1/6$. Sabendo que ocorreu B , o espaço amostral se restringe a $\{2, 4, 6\}$ e $P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{1/6}{1/2} = \frac{1}{3}$. Conhecer B *dobrou* a probabilidade de A — em linguagem do Capítulo 7, o evento B carrega *informação* sobre A .

Eventos independentes: A e B são independentes se conhecer a ocorrência de um não altera a probabilidade do outro:

$$P(A \cdot B) = P(A) \cdot P(B)$$

Generalizando para três eventos, **independência mútua** exige tanto independência par a par ($P(AB) = P(A)P(B)$, etc.) quanto $P(ABC) = P(A)P(B)P(C)$. As duas condições são realmente distintas: existem trios de eventos independentes dois a dois que *não* são mutuamente independentes (exemplo clássico: dois lançamentos de moeda, com $A = \text{"1º é cara"}$, $B = \text{"2º é cara"}$, $C = \text{"os dois resultados são iguais"}$ — verifique que cada par é independente, mas $P(ABC) = 1/4 \neq 1/8$).

i.i.d. (independentes e identicamente distribuídos): termo central em Teoria da Informação (fontes emitem símbolos i.i.d.). São, na verdade, duas propriedades combinadas: *independência* (a probabilidade conjunta é o produto das marginais) e *distribuição idêntica* (todas as variáveis da sequência seguem a mesma distribuição de probabilidade). Fontes reais (como texto em português) **não** são i.i.d. — a letra que vem depois de “q” é quase certamente “u” — e é exatamente essa dependência estatística que compressores como o LZW (Capítulo 9) exploram.

2.3 Função de distribuição, densidade e variáveis discretas/contínuas

Uma variável aleatória pode ser:

- **Discreta:** assume um número contável de valores (ex.: número de caras em 10 lançamentos).
- **Contínua:** assume valores em um conjunto incontável (ex.: um tempo medido em segundos, com precisão arbitrária).

Função de distribuição acumulada (CDF):

$$F(x) = P(X \leq x)$$

com as propriedades: $\lim_{x \rightarrow -\infty} F(x) = 0$, $\lim_{x \rightarrow \infty} F(x) = 1$, F é não-decrescente e contínua à direita.

Para variáveis contínuas, define-se a **função densidade de probabilidade (PDF):**

$$f(x) = \frac{dF(x)}{dx}, \quad F(x) = \int_{-\infty}^x f(u) du, \quad P(a \leq X \leq b) = \int_a^b f(x) dx$$

Ponto conceitual importante (e contraintuitivo para iniciantes): para variáveis contínuas, $P(X = a) = 0$ para qualquer valor específico a — mesmo que a densidade $f(a)$ não seja zero. A densidade descreve *concentração* de probabilidade, não probabilidade em si. Exemplo: a chance de alguém pesar exatamente 70,000000... kg é zero, mas a chance de pesar entre 69 e 71 kg é positiva. Consequência prática: $f(x)$ pode tranquilamente assumir valores **maiores que 1** (basta a densidade estar concentrada num intervalo curto — ex.: Uniforme[0, 0,5] tem $f(x) = 2$). Guarde esse fato: ele explica por que a *entropia diferencial* pode ser negativa (Capítulo 8).

2.4 Valor esperado, momentos, variância

Valor esperado (média):

$$E[X] = \sum_i x_i f(x_i) \text{ (discreto)}, \quad E[X] = \int_{-\infty}^{\infty} x f(x) dx \text{ (contínuo)}$$

O valor esperado é um **operador linear**: $E[aX + bY] = aE[X] + bE[Y]$, *sempre*, mesmo quando X e Y são dependentes — uma propriedade poderosa e frequentemente subestimada. (Já $E[XY] = E[X]E[Y]$ só vale sob independência/descorrelação.)

O valor $E[X^r]$ é chamado de **r -ésimo momento** de X (momento em relação à origem). Os **momentos centrais** são tomados em relação à média: $E[(X - \mu)^r]$. A **variância** é o momento central de segunda ordem:

$$Var(X) = E[(X - \mu)^2] = E[X^2] - (E[X])^2, \quad \sigma = \sqrt{Var(X)} \text{ (desvio padrão)}$$

Os momentos de ordem superior também têm interpretação: o terceiro momento central padronizado mede a **assimetria** (*skewness*) da distribuição, e o quarto mede a **curtose** (peso das caudas) — ambos reaparecem na expansão de Gram-Charlier para estimar entropia (Capítulo 9).

Propriedades da variância:

1. $Var(cX) = c^2 Var(X)$ — escalar a variável multiplica a variância pelo quadrado da escala (ex.: mudar de metros para centímetros multiplica Var por 10 000).
2. $Var(X + k) = Var(X)$ — deslocar (somar constante) não afeta a dispersão.
3. $Var(X) \geq 0$, com igualdade se e somente se X é degenerada (constante).
4. (adicional, muito usada) $Var(X + Y) = Var(X) + Var(Y) + 2 Cov(X, Y)$; sob independência, a covariância zera e as variâncias simplesmente se somam — fato central na prova do Teorema do Limite Central (Capítulo 4).

2.5 Covariância e correlação

$$\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)] = E[XY] - E[X]E[Y]$$

Se X, Y são independentes, $\text{Cov}(X, Y) = 0$ — mas **a recíproca não vale em geral**: covariância nula não implica independência (apenas ausência de dependência *linear*).

Contraexemplo clássico: seja X uniforme em $\{-1, 0, 1\}$ e $Y = X^2$. Então $E[XY] = E[X^3] = 0$ e $E[X] = 0$, logo $\text{Cov}(X, Y) = 0$. Mas Y é *completamente determinada* por X — dependência máxima! A correlação de Pearson enxerga apenas retas; a **informação mútua** (Capítulo 7) foi criada justamente para capturar dependências de *qualquer* forma: neste exemplo, $I(X, Y) > 0$.

Correlação de Pearson (normaliza a covariância pelos desvios-padrão, ficando sempre em $[-1, 1]$):

$$\text{Cor}(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_x \sigma_y}$$

2.6 Mudança de variáveis (Jacobiano)

Quando se transforma um par de variáveis (X_1, X_2) em $(Y_1, Y_2) = (g_1(X_1, X_2), g_2(X_1, X_2))$ de forma invertível, a densidade conjunta se transforma usando o determinante **Jacobiano**:

$$J(x_1, x_2) = \begin{vmatrix} \frac{\partial g_1}{\partial x_1} & \frac{\partial g_1}{\partial x_2} \\ \frac{\partial g_2}{\partial x_1} & \frac{\partial g_2}{\partial x_2} \end{vmatrix}, \quad f_{Y_1, Y_2}(y_1, y_2) = f_{X_1, X_2}(x_1, x_2) |J|^{-1}$$

Intuitivamente, o Jacobiano mede o “fator de distorção de área/volume” que a transformação provoca — é preciso compensar essa distorção para que a densidade continue integrando 1. O caso 1-D mais simples ilustra a ideia: se $Y = aX + b$, então $f_Y(y) = \frac{1}{|a|} f_X\left(\frac{y-b}{a}\right)$.

Exemplo resolvido 1-D: se $X \sim \text{Uniforme}[0, 1]$ e $Y = 2X + 3$, então Y vive em $[3, 5]$ e $f_Y(y) = \frac{1}{2} f_X\left(\frac{y-3}{2}\right) = \frac{1}{2}$ nesse intervalo. A transformação “esticou” o suporte por um fator 2, e a densidade caiu pela metade para compensar — a área total permanece 1.

Se a densidade conjunta se fatora em uma parte dependente só de x e outra só de y , então X e Y são independentes.

2.7 Três desigualdades fundamentais

Estas desigualdades reaparecem repetidamente em Teoria da Informação (por exemplo, na prova de que a divergência KL nunca é negativa — Capítulo 7).

Desigualdade de Jensen

Para uma função **convexa** g e variável aleatória X com esperança finita:

$$E[g(X)] \geq g(E[X])$$

Nota de correção: alguns slides do curso escrevem essa desigualdade com o sinal invertido ($\leq$) — isso corresponderia ao caso de função **côncava**. A forma padrão para função convexa (ex.: $g(x) = x^2$, $g(x) = -\log x$) é $E[g(X)] \geq g(E[X])$; para função côncava (ex.: $g(x) = \log x$), vale o oposto: $E[g(X)] \leq g(E[X])$.

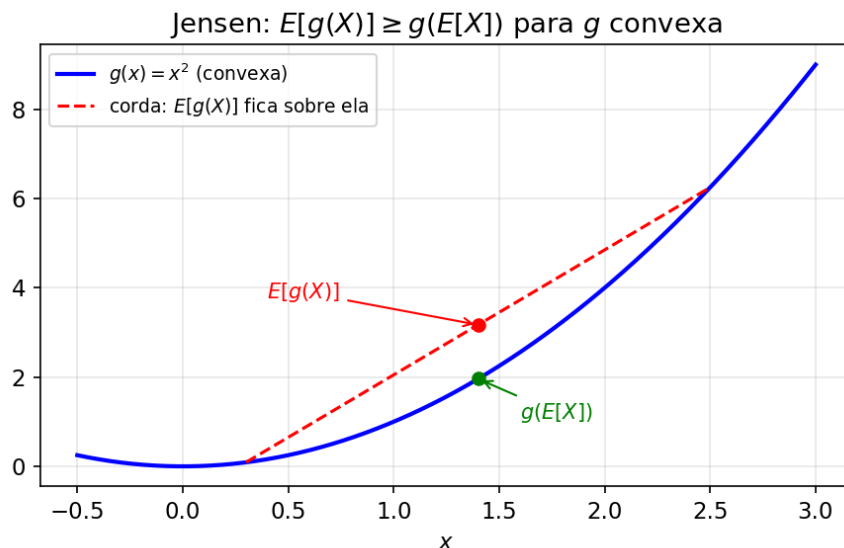


Figura 1: Interpretação geométrica da desigualdade de Jensen para $g(x) = x^2$: o valor médio da função (sobre a corda vermelha) fica acima da função avaliada na média.

Intuição geométrica: para uma função convexa, existe sempre uma reta tangente $h(x) = ax + b$ no ponto $(E[X], g(E[X]))$ tal que $h(x) \leq g(x)$ para todo x (a curva fica sempre “acima” de suas tangentes). Tomando esperança dos dois lados: $E[h(X)] \leq E[g(X)]$. Como h é linear, $E[h(X)] = h(E[X]) = g(E[X])$ — daí a desigualdade.

Exemplo numérico: X vale 1 ou 3, cada um com probabilidade $1/2$, e $g(x) = x^2$. Então $g(E[X]) = 2^2 = 4$, enquanto $E[g(X)] = \frac{1+9}{2} = 5 \geq 4$. A diferença $E[X^2] - (E[X])^2 = 1$ é exatamente a *variância* — Jensen com $g(x) = x^2$ é, na verdade, a prova de que $Var(X) \geq 0$.

Desigualdade de Markov

Para $X \geq 0$ e $a > 0$:

$$P(X \geq a) \leq \frac{E[X]}{a}$$

Só exige que $E[X]$ seja finito — é um limite “grosseiro” (mas válido para qualquer distribuição) da probabilidade de cauda, usando apenas a média. **Exemplo:** se a renda média de uma população é R\$ 3.000, a fração de pessoas ganhando R\$ 30.000 ou mais é *no máximo* 10% — não importa como a renda se distribua.

Desigualdade de Chebyshev

Para X com média μ e variância σ^2 finitas, $k > 0$:

$$P(|X - \mu| \geq k) \leq \frac{\sigma^2}{k^2}$$

Demonstração: aplica-se Markov à variável não-negativa $(X - \mu)^2$ com $a = k^2$: $P((X - \mu)^2 \geq k^2) \leq \frac{E[(X - \mu)^2]}{k^2} = \frac{\sigma^2}{k^2}$; como $(X - \mu)^2 \geq k^2 \iff |X - \mu| \geq k$, chega-se ao resultado. ■

Exemplo prático: com $k = 2\sigma$, a probabilidade de estar a mais de 2 desvios-padrão da média é no máximo $1/4 = 25\%$, para *qualquer* distribuição. Compare com a Normal, em que essa probabilidade é $\approx 4,6\%$: o limite de Chebyshev é folgado, mas é *universal*.

Importância conjunta: Markov e Chebyshev são “*distribution-free*” — não exigem conhecer a forma da distribuição, apenas média (Markov) ou média e variância (Chebyshev). Por isso são conservadoras (os limites que fornecem são folgados), mas extremamente úteis para provar teoremas gerais sem hipóteses fortes sobre a distribuição. **Aplicação (esboço da Lei Fraca dos Grandes Números):** a média amostral $\bar{X}_n$ de n variáveis i.i.d. tem média μ e variância σ^2/n ; por Chebyshev, $P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2} \rightarrow 0$. Esse mesmo estilo de argumento — concentração de médias — está no coração da **Propriedade de Equipartição Assintótica** (AEP), que sustenta os teoremas de Shannon dos Capítulos 9 e 10.

Leituras complementares deste capítulo: ROSS, S. *Probabilidade: Um Curso Moderno com Aplicações* (referência da disciplina); BLITZSTEIN & HWANG, *Introduction to Probability*, 2019 (excepcional em intuição; curso e vídeos gratuitos “Harvard Stat 110”); JAMES, B. R. *Probabilidade: um curso em nível intermediário*, IMPA (rigoroso, em português); CASELLA & BERGER, *Statistical Inference* (caps. 1-4).

Capítulo 3 — Estatística Descritiva e Pré-processamento de Dados

Este capítulo aplica os conceitos de probabilidade a conjuntos de dados observados (amostras), com foco no **pré-processamento** — etapa 2 do pipeline do Capítulo 1. A distinção conceitual importante: o Capítulo 2 fala de quantidades *teóricas* (populacionais), definidas via valor esperado; aqui falamos de suas versões *amostrais*, calculadas a partir de n observações. A ponte entre os dois mundos é a Lei dos Grandes Números: com n crescente, as estatísticas amostrais convergem para as populacionais.

3.1 Medidas de posição

Para um conjunto de valores $\vec{v} = \{v_1, \dots, v_n\}$:

$$\text{Média: } \mu = \frac{1}{n} \sum_{i=1}^n v_i$$

Mediana: valor central da sequência ordenada v' :

$$M = \begin{cases} v'_{(n+1)/2}, & n \text{ ímpar} \\ \frac{1}{2} (v'_{n/2} + v'_{n/2+1}), & n \text{ par} \end{cases}$$

Moda: valor mais frequente.

Exemplo resolvido (dados do curso): quantidades observadas $\{29, 30, 32, 65, 65, 65, 25, 25, 90\}$. Ordenando: 25, 25, 29, 30, 32, 65, 65, 65, 90. Resulta em $\mu = 47,33$; $M = 32$; Moda = 65.

Note como as três medidas **discordam fortemente** (47,3 vs. 32 vs. 65): isso é sintoma de distribuição assimétrica e/ou multimodal, visível no histograma abaixo. Em distribuições simétricas unimodais, média $\approx$ mediana $\approx$ moda; a discrepância entre elas já é, por si só, um diagnóstico exploratório.

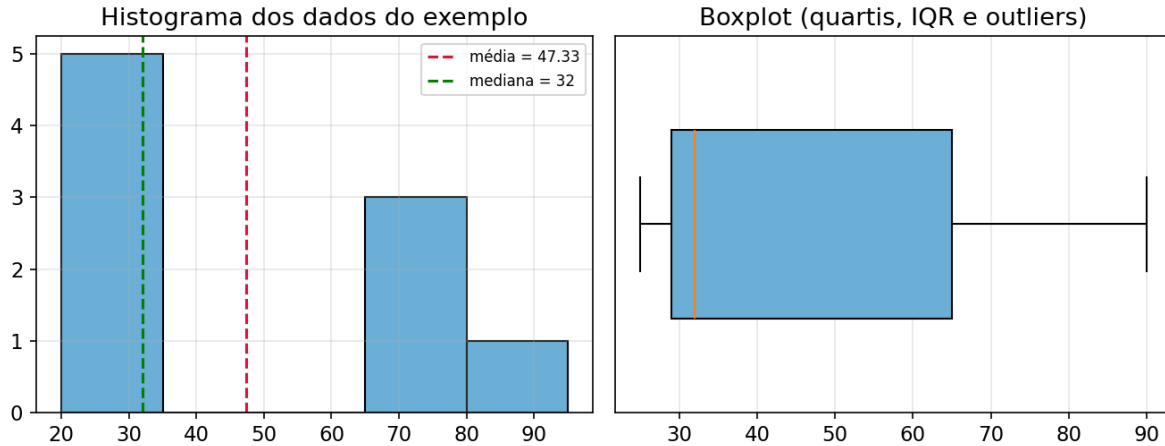


Figura 2: À esquerda, histograma dos dados do exemplo, com média (tracejado vermelho) “puxada” pelos valores altos, distante da mediana (verde). À direita, o boxplot dos mesmos dados.

3.2 Medidas de dispersão

$$\text{Amplitude: } a(\vec{v}) = \max(\vec{v}) - \min(\vec{v})$$

$$\text{Variância (amostral): } \sigma^2 = \frac{1}{n} \sum_{i=1}^n (v_i - \mu)^2; \quad \text{desvio padrão: } \sigma = \sqrt{\sigma^2}$$

Coefficiente de variação de Pearson (dispersão *relativa*, útil para comparar distribuições em escalas diferentes):

$$CV(\vec{v}) = \frac{\sigma(\vec{v})}{\mu(\vec{v})}$$

Exemplo (usando o mesmo conjunto acima): $\sigma^2 \approx 553,6$, $\sigma \approx 23,5$, $CV \approx 0,5$ — a dispersão é da ordem de metade da média, indicando bastante heterogeneidade nos dados.

O CV é **adimensional**, o que permite comparações entre grandezas de naturezas distintas: por exemplo, é possível afirmar que o peso de uma população varia relativamente mais que sua altura mesmo estando em unidades diferentes (kg vs. cm) — comparação impossível com o desvio padrão bruto.

3.3 Distribuição e frequência

$$\text{Frequência relativa de uma faixa } \gamma: f_r(\gamma) = \frac{f(\gamma)}{\sum_i f(v_i)} \times 100.$$

Frequência acumulada: soma das frequências relativas até a faixa corrente: $f_{ar}(\gamma_k) = \sum_{i=1}^k f_r(\gamma_i)$.

Se ordenada de forma decrescente, a distribuição de frequência gera o clássico **Diagrama de Pareto**, usado como indicador de qualidade/priorização (poucos itens concentram a maior parte da frequência/impacto — o famoso “princípio 80/20”). O leitor atento notará a conexão com a **distribuição de Pareto** do Capítulo 4 (cauda pesada) e com o próprio código de Huffman do Capítulo 9: quando poucos símbolos concentram quase toda a probabilidade, a entropia é baixa e há muito espaço para compressão.

3.4 Quartis e percentis

Os quartis dividem o conjunto ordenado em 4 grupos de aproximadamente 25% cada: Q_1 (25º percentil), Q_2 (50º percentil = mediana), Q_3 (75º percentil). A representação visual (**boxplot**, ver figura da seção 3.1) permite ver de uma só vez posição, dispersão, simetria, caudas e *outliers* (valores atípicos, tipicamente fora do intervalo $[Q_1 - 1,5 \cdot IQR, Q_3 + 1,5 \cdot IQR]$, onde $IQR = Q_3 - Q_1$). A regra do $1,5 \times IQR$ foi popularizada por John Tukey em *Exploratory Data Analysis* (1977), livro que fundou a análise exploratória moderna.

3.5 Correlação amostral

$$r_{\vec{v}_1, \vec{v}_2} = \frac{\sum_i (v_{1i} - \mu_1)(v_{2i} - \mu_2)}{\sqrt{\sum_i (v_{1i} - \mu_1)^2} \sqrt{\sum_i (v_{2i} - \mu_2)^2}} = \frac{cov(\vec{v}_1, \vec{v}_2)}{\sigma_1 \sigma_2}, \quad -1 \leq r \leq 1$$

Casos de interpretação:

- **Correlação positiva:** as variáveis crescem/decrescem juntas.
- **Correlação negativa:** uma cresce quando a outra decresce.
- **Correlação nula:** sem padrão *linear* identificável (pode haver dependência não-linear! — reveja o contraexemplo $Y = X^2$ do Capítulo 2).

Esta é a versão amostral (soma sobre observações) da correlação populacional/teórica $Cor(X, Y)$ do Capítulo 2 — formalmente equivalentes, trocando somatório por valor esperado.

Duas advertências clássicas: (i) **correlação não é causalidade** — sorvetes vendidos e afogamentos são correlacionados (ambos causados pelo calor), sem que um cause o outro; (ii) **estatísticas resumo escondem estruturas:** o *quarteto de Anscombe* (1973) exhibe quatro conjuntos de dados com mesma média, variância, correlação ($r = 0,816$) e reta de regressão — mas formas completamente diferentes quando plotados. Moral: **sempre visualize os dados** antes de confiar em números-resumo.

3.6 Recomendações de uso de métricas

Medida	Uso recomendado	Observação
Moda / Frequência	Dados categóricos	—
Percentil	Dados contínuos	—
Média	Localização	Sensível a <i>outliers</i>
Mediana	Localização	Robusta a <i>outliers</i> — preferível na presença deles
Variância / desvio padrão	Dispersão	As mais usadas

Uma prática comum para reduzir o impacto de valores extremos é a **média aparada** (*trimmed mean*): remover os valores mínimo e máximo antes de calcular a média, ou simplesmente preferir a mediana. Para quantificar a sensibilidade: no exemplo da seção 3.1, trocar o valor 90 por 900 (um erro de digitação plausível) arrasta a média de 47,3 para 137,3, enquanto a mediana permanece 32 — é o que a literatura chama de *ponto de ruptura* (breakdown point): 0% para a média, 50% para a mediana.

3.7 Transformação de dados

Antes da análise, dados heterogêneos costumam precisar de ajustes:

- **Discretização:** numérico → categórico (ex.: idade → faixa etária).
- **Codificação (encoding):** categórico → numérico (necessário para muitos algoritmos). Para atributos *nominais*, a técnica adequada é o *one-hot encoding* (um vetor binário por categoria), justamente para não introduzir uma ordem artificial que a escala nominal não possui.
- **Agrupamento por frequência:** reduzir categorias raras a um grupo “outros”.
- **Normalização:** ver seção seguinte — fundamental quando atributos têm escalas muito diferentes.

3.8 Normalização de dados

Motivação: se um atributo varia entre 0 e 1 e outro entre 0 e 1.000.000, algoritmos baseados em distância (KNN, k-means, redes neurais) serão dominados pelo atributo de maior escala, mesmo que ele não seja mais “importante”. A normalização equaliza as escalas.

Exemplo concreto do problema: para classificar clientes usando (renda em R\$, idade em anos), a distância euclidiana entre (5.000, 30) e (5.100, 60) é $\sqrt{100^2 + 30^2} \approx 104,4$ — dominada pela renda: a diferença de 30 anos de idade (enorme!) contribui quase nada. Após z-score, ambas as dimensões pesam de acordo com seus desvios relativos.

Método 1 — Norma constante (normalização local)

Divide cada vetor pela própria norma euclidiana, tornando-o unitário:

$$\vec{x}^* = \frac{\vec{x}}{\|\vec{x}\|}$$

Exemplo: $\vec{x} = (\sqrt{3}, 3, -2)^T$, $\|\vec{x}\| = \sqrt{3+9+4} = \sqrt{16} = 4$, logo $\vec{x}^* = (\sqrt{3}/4, 3/4, -1/2)^T$.

Preserva a direção do vetor, altera apenas o comprimento; depende só das próprias componentes do vetor (por isso “local”). É a normalização típica em recuperação de informação (vetores de documentos), onde o que importa é o *ângulo* entre vetores (similaridade de cosseno), não sua magnitude.

Método 2 — Mudança de escala (min-max, normalização global)

Aplicada variável a variável, exigindo x_{min} e x_{max} dessa variável. Para o intervalo $[0, 1]$:

$$x^* = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Para o intervalo $[-1, +1]$:

$$x^* = 2 \cdot \frac{x - x_{min}}{x_{max} - x_{min}} - 1$$

Cuidado prático: min-max é **frágil a outliers** (um único valor extremo comprime todos os demais num canto do intervalo) e exige atenção em produção — se um dado novo exceder o x_{max} visto no treino, sai do intervalo $[0, 1]$.

Método 3 — Padronização (z-score)

Aplicada variável a variável, resultando em média 0 e variância 1:

$$x^* = \frac{x - \mu}{\sigma_x}, \quad \sigma_x = \sqrt{\frac{\sum_i (x_i - \mu)^2}{N - 1}} \text{ (desvio-padrão amostral, com correção de Bessel } N-1)$$

Exemplo resolvido: dados $\{10, 20, 30\}$: $\mu = 20$, $\sigma_x = \sqrt{\frac{(-10)^2 + 0 + 10^2}{2}} = \sqrt{100} = 10$. Valores padronizados: $\{-1, 0, +1\}$. Interpretação universal do z-score: “quantos desvios-padrão acima/abaixo da média” — o que conecta diretamente com a desigualdade de Chebyshev do Capítulo 2 ($|z| \geq 2$ ocorre com probabilidade $\leq 25\%$ para qualquer distribuição).

Normalização como transformação linear e suas propriedades

Todos os três métodos são casos particulares de uma transformação afim $x^* = ax + b$. Isso traz duas propriedades importantes:

1. **Preserva a forma da distribuição:** como $Y = aX + b$ é linear, se X era gaussiana, Y continua gaussiana — apenas os parâmetros (média, variância) mudam, não a família da distribuição. Formalmente, $F_Y(y) = F_X\left(\frac{y-b}{a}\right)$ para $a > 0$, uma simples reparametrização. (Corolário importante: normalizar **não** transforma dados assimétricos em gaussianos — para isso seriam necessárias transformações *não-lineares*, como log ou Box-Cox.)
2. **Preserva a correlação entre variáveis:** como cada variável é normalizada isoladamente (usando apenas suas próprias estatísticas — min, max, média, desvio), a correlação entre duas variáveis quaisquer não muda. As variáveis normalizadas tornam-se adimensionais, facilitando a comparação.

Leituras complementares deste capítulo: BUSSAB & MORETTIN, *Estatística Básica*, Saraiva (referência em português, caps. 1-5); TUKEY, J. *Exploratory Data Analysis*, 1977; ANSCOMBE, F. “Graphs in Statistical Analysis”, *The American Statistician*, 1973; GÉRON, A. *Mãos à Obra: Aprendizado de Máquina com Scikit-Learn & TensorFlow* (cap. 2 traz o pipeline de pré-processamento completo em código).

Capítulo 4 — Distribuições de Probabilidade Contínuas

Este capítulo cataloga as distribuições contínuas mais usadas no curso — ferramenta essencial para modelar fontes de informação, ruído de canal (Capítulo 10) e técnicas de predição (Capítulo 5). A pergunta prática que este catálogo responde é sempre a mesma: *dado um fenômeno real, qual família de distribuições o modela bem, e com quais parâmetros?*

Distribuição	PDF	$E[X]$	$Var(X)$	Uso típico
Pareto (θ, α)	$\frac{\alpha\theta^\alpha}{x^{\alpha+1}}, x \geq \theta$	$\frac{\alpha\theta}{\alpha-1}$ (se $\alpha > 1$)	existe se $\alpha > 2$	Fenômenos de cauda pesada
Exponencial (λ)	$\lambda e^{-\lambda x}, x \geq 0$	$1/\lambda$	$1/\lambda^2$	Tempo até um evento
Normal (μ, σ^2)	$\frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$	μ	σ^2	Erros de medição, ruído, TLC
Log-normal	$Y = e^X, X \sim N(\mu, \sigma^2)$	$e^{\mu+\sigma^2/2}$	$(e^{\sigma^2} - 1)e^{2\mu+\sigma^2}$	Variáveis econômicas (renda)
Gama (α, λ)	$\frac{\lambda e^{-\lambda x} (\lambda x)^{\alpha-1}}{\Gamma(\alpha)}$	α/λ	α/λ^2	Tempo até n eventos (Poisson)
Beta (a, b)	$\frac{x^{a-1}(1-x)^{b-1}}{B(a,b)}, 0 < x < 1$	$\frac{a}{a+b}$	$\frac{ab}{(a+b)^2(a+b+1)}$	Fenômenos limitados a um intervalo
Weibull (v, α, β)	ver abaixo	—	—	Confiabilidade/vida útil
Uniforme $[a, b]$	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$	Erros pequenos; prior não-informativo

Distribuições contínuas usadas no curso

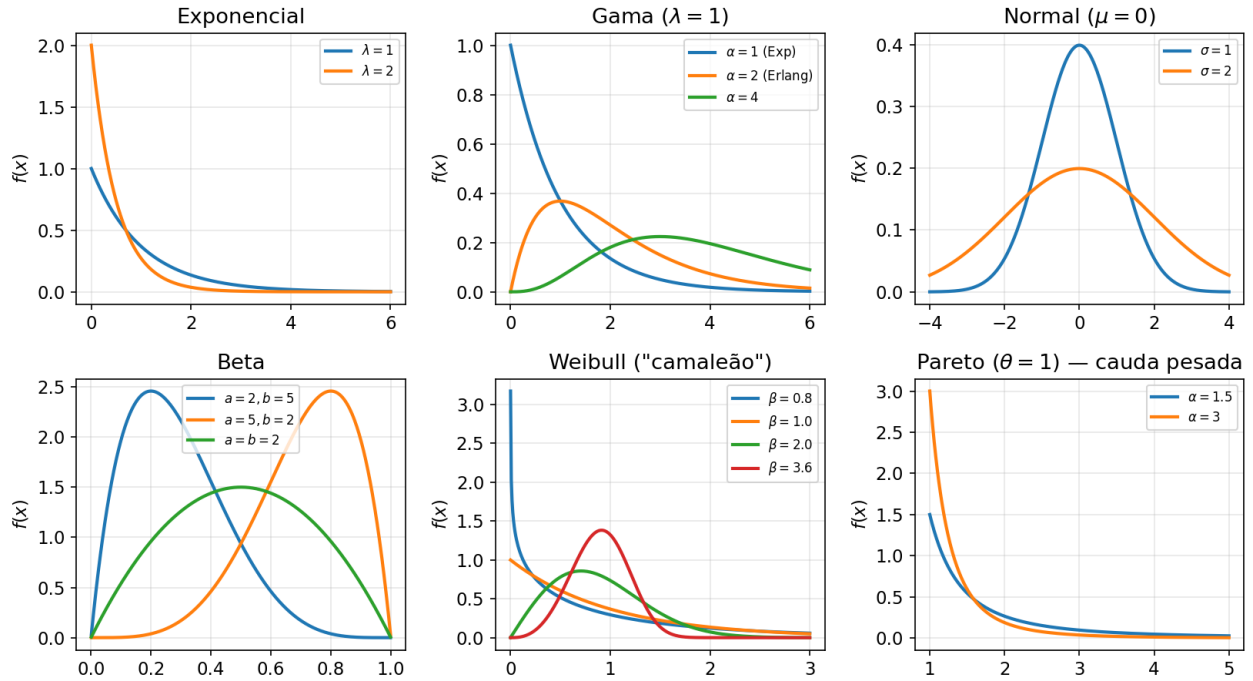


Figura 3: Painei das principais famílias de distribuições contínuas do curso, ilustrando o efeito dos parâmetros de forma e escala.

4.1 Distribuição Exponencial e a propriedade de falta de memória

$$f(x) = \lambda e^{-\lambda x} \quad (x \geq 0), \quad F(a) = 1 - e^{-\lambda a}$$

Propriedade da falta de memória: $P(X > s + t \mid X > s) = P(X > t)$. Se X é o tempo de vida de um equipamento, a chance de durar mais t anos, dado que já durou s anos, é igual à chance de um equipamento novo durar t anos — o equipamento “não envelhece” sob esse modelo.

Demonstração (curta e instrutiva):

$$P(X > s + t \mid X > s) = \frac{P(X > s + t)}{P(X > s)} = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} = e^{-\lambda t} = P(X > t). \blacksquare$$

Notavelmente, a Exponencial é a **única** distribuição contínua com essa propriedade (a análoga discreta é a Geométrica) — por isso ela é o modelo canônico de tempos entre chegadas em filas, tempos entre pacotes em redes e decaimento radioativo.

Exemplo resolvido: o tempo entre requisições a um servidor é exponencial com média de 2 segundos ($\lambda = 0,5 \text{ s}^{-1}$). A probabilidade de a próxima requisição demorar mais de 4 s: $P(X > 4) = e^{-0,5 \cdot 4} = e^{-2} \approx 13,5\%$. Sabendo que já se passaram 3 s sem requisição, a chance de esperar *mais* 4 s continua 13,5% — falta de memória em ação.

4.2 Distribuição Gama e sua relação com Exponencial, Erlang e Qui-quadrado

A função Gama $\Gamma(\alpha) = \int_0^\infty e^{-y} y^{\alpha-1} dy$ satisfaz $\Gamma(\alpha) = (\alpha - 1)\Gamma(\alpha - 1)$ e, para n inteiro, $\Gamma(n) = (n - 1)!$ — ela é, portanto, a extensão contínua do fatorial.

Parâmetros	Nome
$\alpha = 1$	Exponencial
$\alpha = k$ inteiro	Erlang de ordem k (tempo até k eventos de Poisson)
$\alpha = n/2, \lambda = 1/2$	Qui-quadrado com n graus de liberdade

A leitura intuitiva da Erlang ajuda a fixar: se cada evento individual demora um tempo Exponencial(λ), a **soma de k tempos exponenciais independentes** é Gama(k, λ) — ex.: o tempo total para atender 5 clientes numa fila. Na figura do painel (seção anterior), observe como aumentar α desloca a massa para a direita e a torna mais simétrica — prenúncio do Teorema do Limite Central.

4.3 Relação Gama-Beta

Se $X \sim \text{Gama}(\alpha, \lambda)$ e $Y \sim \text{Gama}(\beta, \lambda)$ são independentes, definindo $U = X + Y$ e $V = X/(X + Y)$, então U e V são **independentes**, com $U \sim \text{Gama}(\alpha + \beta, \lambda)$ e $V \sim \text{Beta}(\alpha, \beta)$ — um resultado elegante que se demonstra via mudança de variáveis com Jacobiano $J(x, y) = -\frac{1}{x+y}$ (ver Capítulo 2, seção 2.6).

Interpretação: se X e Y são tempos de duas fases de um processo, o *tempo total* e a *fração do tempo gasta na primeira fase* são independentes — resultado nada óbvio, e a base do uso da Beta como distribuição de proporções (e como *prior* conjugado da Binomial na inferência bayesiana).

4.4 Weibull como distribuição “camaleão”

$$F(x) = 1 - \exp \left\{ - \left(\frac{x - v}{\alpha} \right)^\beta \right\}, \quad x > v$$

O parâmetro de forma β controla a que outra distribuição a Weibull se aproxima:

β	Aproxima
$\beta = 1$	Exponencial (equivalência exata)
$1 < \beta < 3,6$	Log-normal
$\beta \approx 3,6$	Normal

Por essa flexibilidade, é amplamente usada em análise de confiabilidade/vida útil de componentes. Em engenharia de confiabilidade, β tem leitura física direta na “curva da

banheira” de taxas de falha: $\beta < 1$ modela *mortalidade infantil* (falhas precoces, taxa decrescente), $\beta = 1$ modela falhas puramente aleatórias (taxa constante — coerente com a falta de memória da Exponencial) e $\beta > 1$ modela *desgaste* (taxa crescente com a idade).

4.5 Distribuição Normal e o Teorema do Limite Central

Introduzida por DeMoivre (1733) como aproximação da distribuição binomial para n grande. É a base do **Teorema do Limite Central** (a soma de muitas variáveis aleatórias independentes, com variância finita, tende a uma distribuição Normal, *independentemente da distribuição original de cada termo*) — motivo pelo qual aparece tão frequentemente como modelo de ruído em sistemas de comunicação (Capítulo 10) e de erro de medição.

Enunciado um pouco mais preciso: se $X_1, \dots, X_n$ são i.i.d. com média μ e variância σ^2 finita, então

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0, 1) \quad \text{quando } n \rightarrow \infty.$$

O TLC explica por que o **ruído térmico** em eletrônica é gaussiano: ele resulta da soma dos efeitos de um número astronômico de elétrons individuais. Duas observações que anteciparão capítulos futuros: (i) o requisito de *variância finita* é essencial — somas de variáveis de cauda pesada tipo Pareto com $\alpha \leq 2$ **não** convergem para a Normal; (ii) a Normal reaparecerá no Capítulo 8 com um papel de honra na Teoria da Informação: entre todas as distribuições de mesma variância, ela é a de **máxima entropia** — e essa propriedade, não o TLC, é o que a coloca no centro da fórmula de Shannon-Hartley (Capítulo 10).

Leituras complementares deste capítulo: ROSS, S. *Probabilidade: Um Curso Moderno com Aplicações* (caps. 5-6); CASELLA & BERGER, *Statistical Inference* (cap. 3, com o “árvore genealógica” das distribuições); MEYER, P. *Probabilidade: Aplicações à Estatística* (clássico em português); para confiabilidade e Weibull: LEEMIS, L. *Reliability: Probabilistic Models and Statistical Methods*.

Capítulo 5 — Inferência, Processos Estocásticos e Técnicas de Predição

Este capítulo cobre o ferramental estatístico de predição/inferência usado como pano de fundo em aplicações de Big Data e Machine Learning (Parte III).

5.1 Teorema de Bayes

$$P(Y|X) = \frac{P(X|Y) P(Y)}{P(X)}, \quad P(X) = \sum_Y P(X, Y)$$

Permite atualizar a crença sobre Y (**probabilidade a posteriori**) após observar evidência X , a partir de uma crença inicial $P(Y)$ (**a priori**) e do modelo de **verossimilhança** $P(X|Y)$.

Exemplo resolvido (teste diagnóstico — o exemplo mais importante de Bayes): uma doença atinge 1% da população ($P(D) = 0,01$). Um teste tem sensibilidade de 99% ($P(+|D) = 0,99$) e taxa de falso positivo de 5% ($P(+|\bar{D}) = 0,05$). Uma pessoa testou positivo — qual a chance de estar doente?

$$P(D|+) = \frac{P(+|D)P(D)}{P(+|D)P(D) + P(+|\bar{D})P(\bar{D})} = \frac{0,99 \times 0,01}{0,99 \times 0,01 + 0,05 \times 0,99} = \frac{0,0099}{0,0594} \approx 16,7\%$$

Apenas $\approx 17\%$! A intuição da maioria das pessoas (inclusive médicos, em estudos famosos de Gigerenzer) erra por confundir $P(D|+)$ com $P(+|D)$ — a chamada *falácia da condicional invertida*. O prior baixo (1%) domina: em 10.000 pessoas, há ~ 99 verdadeiros positivos contra ~ 495 falsos positivos. Este raciocínio de “atualizar crenças pela evidência” é exatamente o mecanismo do *modelo generativo* da seção 5.3 e do classificador Naive Bayes usado em filtros de spam.

5.2 Processos estocásticos e Cadeias de Markov

Um **processo estocástico** é uma família de variáveis aleatórias indexadas por um parâmetro (tipicamente o tempo): $X(t, \omega)$.

Propriedade de Markov: dado o presente, o futuro é independente do passado:

$$P(X_{n+1} = x_{n+1} \mid X_0, \dots, X_n) = P(X_{n+1} = x_{n+1} \mid X_n)$$

A **matriz de transição** $P = (p_{ij})$, com $p_{ij} = P(X_{n+1} = j \mid X_n = i)$, é estocástica ($\sum_j p_{ij} = 1$). A **equação de Chapman-Kolmogorov** permite compor transições de múltiplos passos: $P_{ij}(m+n) = \sum_k P_{ik}(m)P_{kj}(n)$, ou seja, $P^{(n)} = P^n$ (a matriz de transição em n passos é a n -ésima potência da matriz de um passo).

Exemplo resolvido — previsão do tempo (0 = chove, 1 = não chove):

Cadeia de Markov do exemplo de previsão do tempo

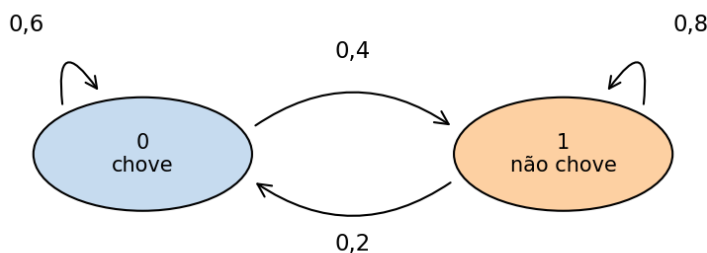


Figura 4: Diagrama de estados da cadeia de Markov do exemplo. As setas carregam as probabilidades de transição; note que as saídas de cada estado somam 1.

$$P = \begin{pmatrix} 0,6 & 0,4 \\ 0,2 & 0,8 \end{pmatrix} \Rightarrow P^2 = \begin{pmatrix} 0,44 & 0,56 \\ 0,28 & 0,72 \end{pmatrix} \Rightarrow P^4 = \begin{pmatrix} 0,3504 & 0,6496 \\ 0,3248 & 0,6752 \end{pmatrix}$$

Logo, partindo de um dia chuvoso, a chance de chover novamente daqui a 4 dias é $p_{00}^{(4)} = 0,3504$.

Cadeia ergódica: se toda transição $P_{ij}^{(m)} > 0$ para algum m , a cadeia converge para uma **distribuição estacionária** π que não depende do estado inicial. No exemplo acima, observe as linhas de P^4 ficando parecidas — a cadeia está convergindo. A estacionária resolve $\pi P = \pi$: aqui, $\pi_0 \cdot 0,4 = \pi_1 \cdot 0,2$ com $\pi_0 + \pi_1 = 1$ dá $\pi = (1/3, 2/3)$ — no longo prazo, chove em 1/3 dos dias, *qualquer que tenha sido o clima inicial*.

Dois aplicações célebres de cadeias de Markov: (i) o **PageRank** original do Google modela um “navegador aleatório” sobre o grafo da web; a relevância de

uma página é sua probabilidade na distribuição estacionária; (ii) o próprio **Shannon (1948)** usou cadeias de Markov para modelar a língua inglesa — gerando texto sintético com dependências de ordem crescente — e definiu a *taxa de entropia* de uma fonte com memória, $H(\mathcal{X}) = \lim_n \frac{1}{n} H(X_1, \dots, X_n)$, generalizando a entropia i.i.d. do Capítulo 6.

5.3 Teoria da decisão

Para prever uma saída (contínua = **regressão**; categórica = **classificação**) a partir de uma entrada x , três abordagens são possíveis:

1. **Modelo generativo**: estima $P(x|C_k)$ e $P(C_k)$ separadamente e aplica Bayes: $P(C_k|x) = \frac{P(x|C_k)P(C_k)}{P(x)}$. Mais custoso, mas permite gerar dados sintéticos e detectar *outliers*.
2. **Modelo discriminativo**: estima diretamente $P(C_k|x)$, sem passar pela distribuição conjunta.
3. **Função discriminante**: aprende diretamente uma função $f(x)$ que mapeia x a uma classe, sem calcular probabilidades — mais simples e barato, mas perde informação útil (ex.: para rejeitar decisões de baixa confiança).

Exemplos de cada família, para ancorar: Naive Bayes e GMM são *generativos*; regressão logística é *discriminativa*; SVM e a maior parte das redes neurais com decisão por argmax operam como *funções discriminantes*. Os modernos modelos de linguagem e de difusão de imagens são, no sentido acima, gigantescos modelos *generativos*.

A **regra de decisão de erro mínimo**: classificar x na classe de maior $P(C_k|x)$ minimiza a probabilidade de erro. Quando os erros têm custos diferentes, usa-se uma **matriz de perda** L_{kj} e minimiza-se a *perda esperada* em vez da taxa de erro bruta.

Exemplo: em triagem de câncer, um falso negativo (doença não detectada) custa muito mais que um falso positivo (exame extra desnecessário); a matriz de perda desloca o limiar de decisão para longe de 0,5, aceitando mais falsos positivos para reduzir falsos negativos — este é o mesmo trade-off precisão × recall do Capítulo 14.

5.4 Técnicas de predição (visão panorâmica)

Técnica	Ideia central
Mínimos Quadrados	$\hat{\beta} = (X^T X)^{-1} X^T Y$ — minimiza a soma dos erros ao quadrado
KNN (k-vizinhos)	Prediz pela média dos k pontos mais próximos; sensível à escala (daí a importância da normalização, Cap. 3)
MLE (Máxima Verossimilhança)	Escolhe os parâmetros que tornam os dados observados mais prováveis; não-viesado e consistente, porém custoso

Técnica	Ideia central
Regularização (Ridge/Lasso)	Penaliza coeficientes grandes; Ridge (penalidade L_2) encolhe coeficientes, Lasso (penalidade L_1) zera coeficientes (seleção de variáveis)
LARS	Variante eficiente do <i>forward stepwise</i> , intimamente ligada ao Lasso
Regressão local / Kernel	Ajusta um modelo separado para cada ponto-alvo, ponderando vizinhos por uma função kernel
Regressão Bayesiana	Trata os parâmetros como variáveis aleatórias com prior/posterior Gaussianos
Monte Carlo / Importance Sampling	Aproxima integrais difíceis (como $E[f]$) por amostragem
Modelos de Mistura Finita	População = combinação de K subgrupos com distribuições próprias — base teórica do <i>clustering</i> probabilístico (GMM)

Duas conexões dessa tabela com a Teoria da Informação merecem destaque: (i) maximizar a verossimilhança (MLE) é *matematicamente equivalente* a minimizar a divergência KL entre a distribuição empírica dos dados e o modelo (Capítulo 7) — MLE “aproxima o modelo da realidade” no sentido informacional; (ii) sob ruído gaussiano, MLE reduz-se exatamente aos Mínimos Quadrados — o MSE não é uma escolha arbitrária, é a consequência da hipótese de erro Normal (Capítulo 4).

Essas técnicas reaparecem, em versão simplificada, no Capítulo 14 (treinamento de redes neurais) — o mínimo quadrado, por exemplo, é exatamente a função de perda (MSE) usada para treinar um modelo de regressão linear via gradiente descendente.

Leituras complementares deste capítulo: BISHOP, C. *Pattern Recognition and Machine Learning* (caps. 1-3, referência da disciplina); NORRIS, J. *Markov Chains*, Cambridge (rigoroso e legível); HASTIE, TIBSHIRANI & FRIEDMAN, *The Elements of Statistical Learning* (caps. 2-3, gratuito no site dos autores); GIGERENZER, G. *Calculated Risks*, 2002 (sobre a interpretação de Bayes em diagnóstico); PAGE, L. et al. “The PageRank Citation Ranking”, Stanford, 1999.

PARTE II – TEORIA DA INFORMAÇÃO CLÁSSICA (SHANNON)

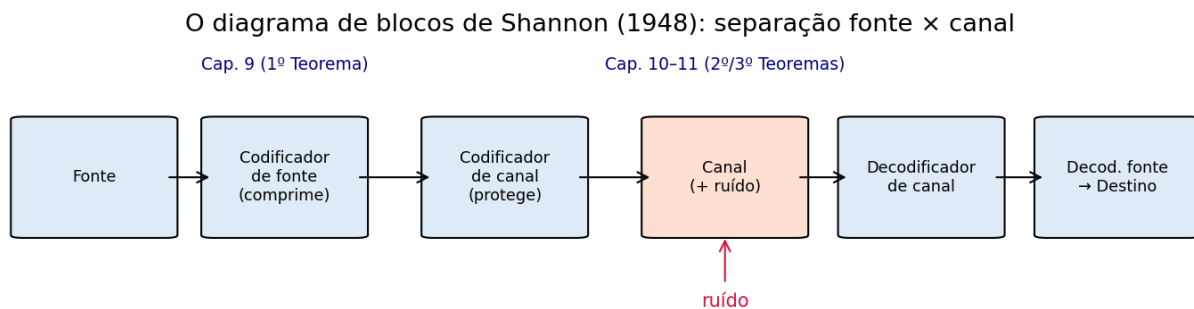


Figura 5: O paradigma de Shannon (1948): a comunicação se decompõe em compressão (codificação de fonte) e proteção contra ruído (codificação de canal). Este diagrama organiza os Capítulos 9, 10 e 11.

Capítulo 6 — Quantidade de Informação e Entropia

Chegamos ao núcleo conceitual da disciplina. A pergunta central da Teoria da Informação, formulada por Claude Shannon em seu artigo seminal de 1948 (“*A Mathematical Theory of Communication*”), é:

Quanta informação uma variável aleatória contém, quando observamos um valor específico dela?

Vale registrar o impacto do artigo: em cerca de 50 páginas, Shannon definiu entropia, provou os teoremas de codificação de fonte e de canal, e criou de uma só vez o arcabouço matemático de toda a comunicação digital — do ZIP ao 5G, do CD ao Wi-Fi. Poucas vezes na história uma única publicação fundou uma disciplina inteira já em estado maduro.

6.1 Autoinformação (informação de um evento)

Considere uma fonte discreta com alfabeto $A = \{a_0, a_1, \dots, a_{K-1}\}$ e distribuição de probabilidade $P(A = a_k) = p_k$, com $\sum_k p_k = 1$.

A informação pode ser entendida como um **nível de surpresa**: um evento improvável nos surpreende mais (carrega mais informação); um evento certo não surpreende ninguém (carrega informação zero). Isso motiva a definição:

$$I(a_k) = \log_{\alpha} \left(\frac{1}{p_k} \right) = -\log_{\alpha}(p_k)$$

A base α do logaritmo define a unidade: $\alpha = 2 \rightarrow$ **bits**; $\alpha = e \rightarrow$ nats; $\alpha = 10 \rightarrow$ hartleys/dits. Salvo indicação contrária, este material usa $\log_2$ (bits).

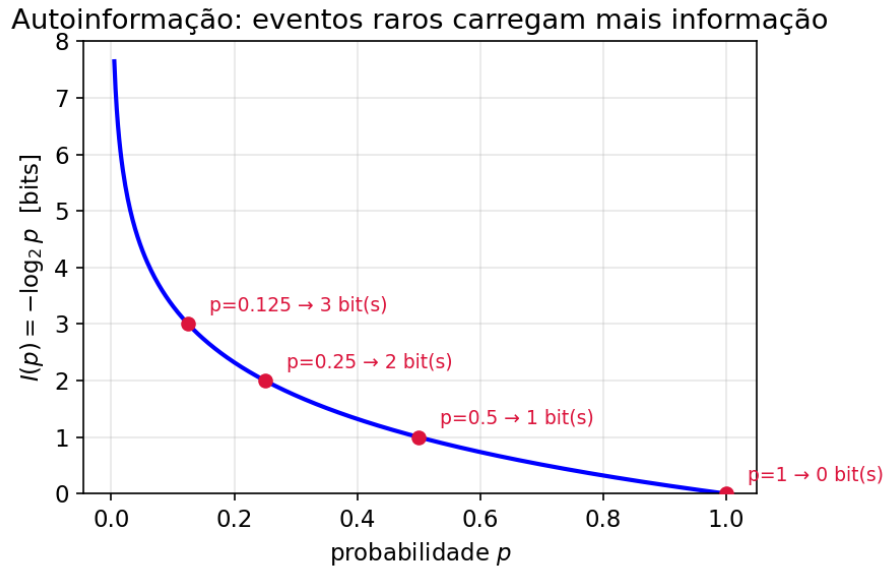


Figura 6: A autoinformação como função da probabilidade do evento. Cada vez que a probabilidade cai pela metade, a informação cresce exatamente 1 bit.

Propriedades da autoinformação:

1. $I(a_k) = 0$ se $p_k = 1$ (evento certo não traz novidade).
2. $I(a_k) \geq 0$ para $0 \leq p_k \leq 1$ (informação nunca é negativa).
3. $I(a_k) > I(a_i)$ se $p_k < p_i$ (eventos mais raros carregam mais informação).
4. $I(a_k a_i) = I(a_k) + I(a_i)$ se a_k e a_i são independentes (informação de eventos independentes é aditiva).

A propriedade 4 é justamente a razão de usarmos o logaritmo: para que a informação de eventos conjuntos independentes seja a soma das informações individuais, é necessário que $I(p_k \cdot p_i) = I(p_k) + I(p_i)$ — e essa é exatamente a propriedade fundamental do logaritmo, $\log(xy) = \log x + \log y$. Pode-se provar (e Shannon provou) que, junto com continuidade e monotonicidade, essas exigências determinam a forma $-\log p$ **de maneira única**, a menos da escolha da base.

A curva $I(p) = -\log_2(p)$ no intervalo $(0, 1]$ é estritamente decrescente e convexa: tende a infinito quando $p \rightarrow 0$ e vale exatamente 0 quando $p = 1$.

Exemplos numéricos de calibração da intuição:

- Uma moeda honesta: $I = -\log_2(1/2) = 1$ bit por lançamento.
- Uma carta específica de um baralho de 52: $I = \log_2 52 \approx 5,7$ bits.
- “Saiu 13 na Mega-Sena entre 60 dezenas”: $\log_2 60 \approx 5,9$ bits; a sena completa (uma combinação entre $\binom{60}{6} \approx 5 \times 10^7$): $\approx 25,6$ bits.
- A letra “a” em português ($p \approx 0,146$): $\approx 2,8$ bits; a letra “z” ($p \approx 0,004$): ≈ 8 bits — letras raras “dizem mais” quando aparecem (pense no jogo da forca).

6.2 Entropia de Shannon

A **entropia** é a média (valor esperado) da autoinformação sobre todos os símbolos da fonte:

$$H(A) = \sum_k p_k \cdot I(a_k) = - \sum_k p_k \log_{\alpha}(p_k) = -E\{\log(p_A(a))\}$$

Interpretação: $H(A)$ mede a quantidade **média** de informação (ou incerteza) por símbolo emitido pela fonte. Também pode ser lida como o “custo médio em bits” (quando $\alpha = 2$) necessário para representar cada símbolo — essa leitura é a que conecta entropia a compressão de dados (Capítulo 9).

Propriedades:

$$0 \leq H(A) \leq \log_{\alpha} K$$

onde $K = |A|$ é o tamanho do alfabeto.

- **Mínimo** ($H = 0$): quando um símbolo tem $p_k = 1$ e os demais têm probabilidade zero — não há incerteza, a fonte é totalmente determinística.
- **Máximo** ($H = \log_{\alpha} K$): quando todos os símbolos são **equiprováveis** ($p_k = 1/K$) — máxima incerteza possível. (A prova do máximo é uma aplicação direta da desigualdade de Jensen/da não-negatividade da divergência KL — Capítulo 7.)

Exemplo resolvido (dado viciado): um dado tem $P(6) = 1/2$ e as demais faces com $1/10$ cada.

$$H = -\frac{1}{2} \log_2 \frac{1}{2} - 5 \cdot \frac{1}{10} \log_2 \frac{1}{10} = 0,5 + 0,5 \times 3,322 \approx 2,16 \text{ bits}$$

Compare com o dado honesto: $\log_2 6 \approx 2,585$ bits. O vício *reduziu* a incerteza em $\approx 0,42$ bit por lançamento — e o Capítulo 9 mostrará que isso significa, literalmente, que a sequência de lançamentos do dado viciado pode ser comprimida nessa proporção.

Exemplo real (a entropia das línguas): tratando o inglês como fonte de 27 símbolos (26 letras + espaço), a entropia máxima seria $\log_2 27 \approx 4,75$ bits/letra. Usando as frequências reais das letras, cai para $\approx 4,1$; considerando dependências entre letras (digramas, trigramas, palavras), Shannon estimou experimentalmente algo entre **0,6 e 1,3 bit por letra** (Shannon, 1951, “Prediction and Entropy of Printed English”). Ou seja: um texto em linguagem natural é $\sim 75\%$ redundante — é essa redundância que permite ler “vc cnsq entndr isto sm as vgais” e que os compressores do Capítulo 9 exploram.

Caso binário

Para uma fonte com dois símbolos, p_0 e $p_1 = 1 - p_0$:

$$H(A) = -p_0 \log_2 p_0 - (1 - p_0) \log_2 (1 - p_0) \equiv H_b(p_0)$$

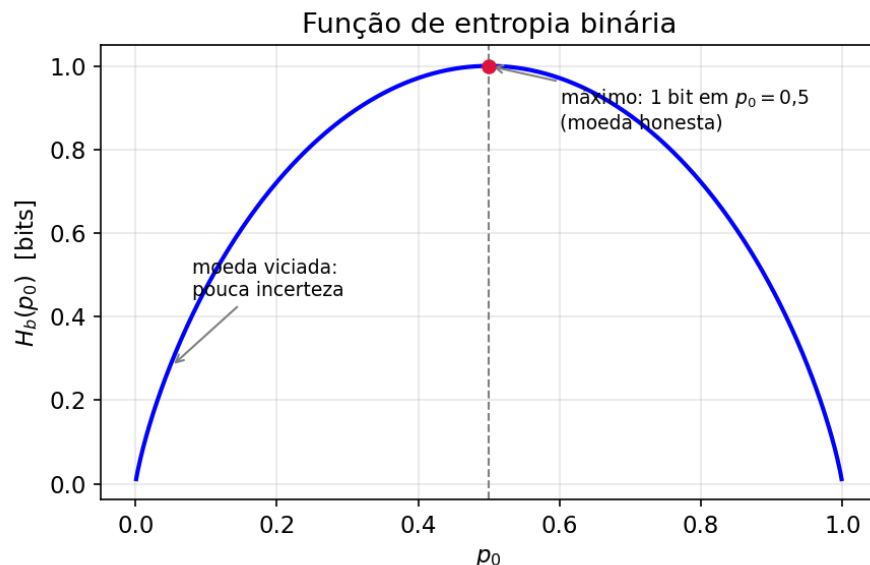


Figura 7: A função de entropia binária: cônica, nula nos extremos e máxima (1 bit) para a moeda honesta.

Interpretação: uma moeda “viciada” (p_0 perto de 0 ou 1) tem pouca incerteza (poucas “surpresas”); uma moeda “honesta” ($p_0 = 0,5$) tem incerteza máxima. Guarde a forma dessa curva: ela reaparecerá *invertida* na capacidade do canal binário simétrico, $C = 1 - H_b(p)$ (Capítulo 10).

6.3 Extensão de uma fonte discreta sem memória

Se agruparmos n símbolos consecutivos de uma fonte sem memória em um “superssímbolo”, o alfabeto estendido A^n tem K^n blocos possíveis. Como os símbolos originais são estatisticamente independentes:

$$P(s[A^n]) = \prod_{i=1}^n P(s_i[A]) \implies H(A^n) = n \cdot H(A)$$

Esse resultado, aparentemente simples, é a base teórica para a **codificação por blocos** e para o **Primeiro Teorema de Shannon** (Capítulo 9): quanto maior o bloco, mais próxima a taxa de codificação pode chegar da entropia teórica da fonte.

Verificação numérica: fonte binária com $p_0 = 0,9$, $H(A) \approx 0,469$ bit. A extensão de ordem 2 tem símbolos $\{00, 01, 10, 11\}$ com probabilidades $\{0,81; 0,09; 0,09; 0,01\}$ e entropia $H(A^2) = -0,81 \log_2 0,81 - 2(0,09 \log_2 0,09) - 0,01 \log_2 0,01 \approx 0,938 = 2 \times 0,469$. ✓ — e um Huffman sobre esses 4 blocos já comprime melhor que qualquer código símbolo a símbolo sobre o alfabeto original.

Leituras complementares deste capítulo: SHANNON, C. “A Mathematical Theory of Communication”, 1948 (disponível gratuitamente; a Parte I é

surpreendentemente legível); SHANNON, C. “Prediction and Entropy of Printed English”, 1951; COVER & THOMAS, *Elements of Information Theory* (cap. 2); MACKAY, D. *Information Theory, Inference, and Learning Algorithms*, 2003 (gratuito em inference.org.uk — caps. 1-4, com exercícios brilhantes); PIERCE, J. *An Introduction to Information Theory: Symbols, Signals and Noise* (introdução acessível, ed. Dover).

Capítulo 7 — Entropia Conjunta, Condicional, Divergência KL e Informação Mútua

Este capítulo estende a entropia para *pares* de variáveis. A pergunta motivadora: se observo B , quanto isso me ensina sobre A ? A resposta rigorosa — a informação mútua — é possivelmente o conceito mais reutilizado de todo o curso: ela define a capacidade de canal (Capítulo 10), o ganho de informação em árvores de decisão (seção 7.6) e várias funções de perda de Machine Learning.

7.1 Entropia conjunta

Estende a entropia para duas variáveis aleatórias, medindo a incerteza total de um sistema composto:

$$H(A, B) = - \sum_{a \in A} \sum_{b \in B} p(a, b) \log[p(a, b)] = -E\{\log[p(a, b)]\}$$

Exemplo (moeda honesta + dado honesto de 6 faces, independentes): há 12 combinações equiprováveis, cada uma com probabilidade $1/12$:

$$H(X, Y) = - \sum_{i=1}^{12} \frac{1}{12} \log_2 \frac{1}{12} = \log_2(12) \approx 3,585 \text{ bits}$$

Note que $3,585 = \log_2 2 + \log_2 6 = H(X) + H(Y)$: sob independência, as entropias **somam** — coerente com a propriedade 4 da autoinformação (Capítulo 6). Quando há dependência, $H(A, B) < H(A) + H(B)$: parte da incerteza é “compartilhada”, e essa sobreposição é exatamente a informação mútua (seção 7.5).

7.2 Entropia condicional

Mede a incerteza **remanescente** sobre A depois de conhecer B :

$$H(A|B) = \sum_b p(b) H(A|B=b) = - \sum_b \sum_a p(a,b) \log[p(a|b)] = -E\{\log[p(a|b)]\}$$

No exemplo moeda + dado: como as variáveis são independentes, conhecer o dado não ajuda a prever a moeda: $H(\text{moeda} | \text{dado}) = H(\text{moeda}) = 1 \text{ bit}$.

Atenção à estrutura da definição: $H(A|B)$ é uma **média ponderada** das entropias condicionais $H(A|B=b)$ sobre os valores possíveis de b . Para um valor *específico* b , pode ocorrer $H(A|B=b) > H(A)$ (uma observação particular pode *aumentar* a confusão); mas **em média** condicionar nunca aumenta a incerteza: $H(A|B) \leq H(A)$ (seção 8.2).

7.3 Regra da cadeia

$$H(A, B) = H(A) + H(B|A) = H(B) + H(A|B)$$

A incerteza total de um sistema conjunto é a incerteza de uma parte mais a incerteza remanescente da outra parte, dado que a primeira já é conhecida. Generaliza-se para n variáveis:

$$H(A_1, \dots, A_N) = \sum_{i=1}^N H(A_i | A_{i-1}, \dots, A_1)$$

A prova de duas variáveis cabe em uma linha e é instrutiva: $p(a, b) = p(a)p(b|a)$, logo $-\log p(a, b) = -\log p(a) - \log p(b|a)$; tomando o valor esperado dos dois lados, obtém-se a regra. É a versão “em logaritmo” da regra do produto de probabilidades.

7.4 Divergência de Kullback-Leibler (entropia relativa / entropia cruzada)

Mede a “distância” (não simétrica) entre duas distribuições p e g — mais precisamente, a **ineficiência de assumir** que uma variável segue $p(x)$ quando a verdadeira distribuição é $g(x)$:

$$D(p\|g) = \sum_x p(x) \log \left(\frac{p(x)}{g(x)} \right) = E_p \left\{ \log \frac{p(x)}{g(x)} \right\}$$

Propriedades:

1. $D(p\|g) \geq 0$, com igualdade se e somente se $p = g$ (consequência direta da desigualdade de Jensen aplicada à função convexa $-\log(\cdot)$ — ver Capítulo 2). Esse resultado tem nome próprio — **desigualdade de Gibbs** — e é a “faca suíça” das provas em Teoria da Informação: dele decorrem $H \leq \log K$, $I \geq 0$ e $H(A|B) \leq H(A)$.

2. **Não é simétrica:** $D(p\|g) \neq D(g\|p)$ em geral — por isso, apesar de ser chamada informalmente de “distância”, não é uma métrica no sentido matemático estrito (não satisfaz simetria nem, em geral, a desigualdade triangular).
3. Invariante a permutação, escalonamento de amplitude e transformações monotônicas não-lineares das componentes.

Exemplo resolvido: modelo $g = (1/2, 1/2)$ para uma moeda cuja distribuição real é $p = (3/4, 1/4)$:

$$D(p\|g) = \frac{3}{4} \log_2 \frac{3/4}{1/2} + \frac{1}{4} \log_2 \frac{1/4}{1/2} = \frac{3}{4}(0,585) + \frac{1}{4}(-1) \approx 0,189 \text{ bit.}$$

Interpretação operacional (via Capítulo 9): quem projetar um compressor assumindo a moeda honesta gastará, em média, 0,189 bit **a mais** por símbolo do que o ótimo. Invertendo os papéis, $D(g\|p) \approx 0,208 \text{ bit} \neq 0,189$ — a assimetria em ação.

Vocabulário relacionado (muito usado em ML): a *entropia cruzada* é $H(p, g) = -\sum_x p(x) \log g(x) = H(p) + D(p\|g)$. Como $H(p)$ é fixa (não depende do modelo g), **minimizar a entropia cruzada equivale a minimizar a KL** — é por isso que a *cross-entropy loss* dos classificadores neurais (Capítulo 14) é, no fundo, “aproxime sua distribuição prevista da distribuição real dos rótulos”. A KLD também é componente central em modelos generativos como VAEs.

7.5 Informação Mútua

Definida como a divergência KL entre a distribuição conjunta e o produto das marginais:

$$I(A, B) = \sum_{a,b} p(a, b) \log \left(\frac{p(a, b)}{p(a)p(b)} \right) = D(p(a, b) \| p(a)p(b))$$

Ou seja: I mede o quanto a conjunta *se afasta* do que ela seria se as variáveis fossem independentes.

Relações fundamentais:

$$I(A, B) = H(A) - H(A|B) = H(B) - H(B|A) = H(A) + H(B) - H(A, B)$$

A informação mútua mede **quanto conhecer B reduz a incerteza sobre A** (e vice-versa, por simetria). Se A e B são independentes, $I(A, B) = 0$; quanto mais dependentes, maior $I(A, B)$. Como caso particular, $I(A, A) = H(A)$ (conhecer A elimina toda a incerteza sobre si mesma, pois $H(A|A) = 0$).

Relações entre entropias e informação mútua

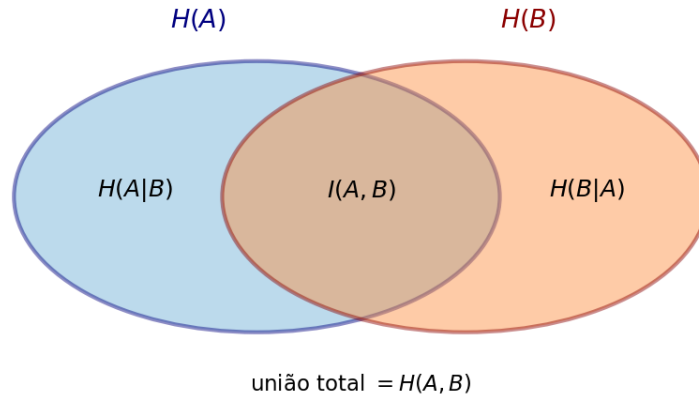


Figura 8: O “diagrama de Venn” informacional: os círculos são $H(A)$ e $H(B)$; a interseção é $I(A, B)$; as luas laterais são as entropias condicionais; a união é $H(A, B)$.

Exemplo resolvido completo (canal binário ruidoso em miniatura): um bit A equiprovável é transmitido e chega como B , invertido com probabilidade 0,1. A distribuição conjunta:

$p(a, b)$	$b = 0$	$b = 1$
$a = 0$	0,45	0,05
$a = 1$	0,05	0,45

- $H(A) = H(B) = 1$ bit (marginais equiprováveis).
- $H(A, B) = -2(0,45 \log_2 0,45) - 2(0,05 \log_2 0,05) \approx 1,469$ bits.
- $H(A|B) = H(A, B) - H(B) \approx 0,469$ bit — a incerteza que o ruído deixa sobre o bit enviado.
- $I(A, B) = H(A) - H(A|B) \approx 0,531$ bit.

Ou seja: cada uso desse canal entrega, em média, só 0,531 bit de informação útil por bit transmitido. Observe que $0,531 = 1 - H_b(0,1)$: acabamos de calcular, sem anunciar, a **capacidade do canal binário simétrico** com $p = 0,1$ — fórmula que o Capítulo 10 apresentará oficialmente.

7.6 Ganho de Informação e árvores de decisão

Em Machine Learning, a informação mútua reaparece com outro nome: **Ganho de Informação** (*Information Gain*):

$$IG(Y|X) = H(Y) - H(Y|X) \equiv I(X, Y)$$

É o critério-chave para escolher, em algoritmos de árvore de decisão (ex.: C4.5), qual atributo X usar para dividir os dados: escolhe-se o atributo que **mais reduz a incerteza** sobre a variável-alvo Y .

Exemplo resolvido: 10 e-mails, 5 spam e 5 legítimos: $H(Y) = 1$ bit. O atributo $X =$ “contém a palavra ‘grátis?’” divide: entre os 4 que contêm, 4 são spam ($H = 0$); entre os 6 restantes, 1 é spam ($H(1/6) \approx 0,65$). Então $H(Y|X) = 0,4 \cdot 0 + 0,6 \cdot 0,65 = 0,39$ e $IG = 1 - 0,39 = 0,61$ bit. Um atributo alternativo que dividisse os e-mails em 5/5 com 50% de spam em cada lado teria $IG = 0$ — inútil. A árvore escolhe “grátis” para a raiz.

Limitação: o ganho de informação tende a favorecer atributos com muitos valores distintos (mesmo que irrelevantes — um atributo “ID do e-mail”, com um valor único por exemplo, zera $H(Y|X)$ e parece perfeito, mas não generaliza) — por isso existem alternativas como **Gain Ratio** (normaliza o IG pela entropia do próprio atributo, usado no C4.5) e **Índice de Gini** (usado no algoritmo CART).

Leituras complementares deste capítulo: COVER & THOMAS, *Elements of Information Theory* (cap. 2 — as provas completas de todas as desigualdades aqui citadas); MACKAY, *Information Theory, Inference, and Learning Algorithms* (caps. 8-9); KULLBACK & LEIBLER, “On Information and Sufficiency”, *Annals of Mathematical Statistics*, 1951; QUINLAN, J. R. *C4.5: Programs for Machine Learning*, 1993; para a versão contínua/estimada da informação mútua em ML: KRASKOV et al., “Estimating Mutual Information”, *Physical Review E*, 2004.

Capítulo 8 — Entropia Diferencial e Generalizações da Entropia

8.1 Entropia diferencial

Quando a variável é contínua, a entropia recebe o nome de **entropia diferencial**:

$$H(X) = - \int_S p_X(x) \log[p_X(x)] dx$$

onde S é o suporte da variável. Diferença importante em relação ao caso discreto: **a entropia diferencial pode ser negativa** — não existe o limite inferior $H \geq 0$ que vale no caso discreto, pois a densidade $p_X(x)$ pode assumir valores maiores que 1 (veja a seção 2.3).

Exemplo mínimo que exhibe o fenômeno (Uniforme): para $X \sim \text{Uniforme}[0, a]$, $H(X) = \log_2 a$. Com $a = 4$: $H = 2$ bits; com $a = 1$: $H = 0$; com $a = 1/2$: $H = -1$ bit — negativa! A leitura correta: a entropia diferencial não conta “bits absolutos”, mas mede incerteza *em relação à escala do contínuo*; descrever um real exige, a rigor, infinitos bits, e H diferencial captura apenas o termo que varia com a “largura” da distribuição. (Formalmente: quantizando X com passo Δ , a entropia discreta é $\approx H(X) + \log_2(1/\Delta)$ — o termo divergente $\log_2(1/\Delta)$ é o custo de precisão, e $H(X)$ é a parte que depende da distribuição.)

Exemplo resolvido — entropia diferencial da Normal ($X \sim N(0, \sigma^2)$, em nats):

$$H[p_X(x)] = - \int \phi(x) \ln[\phi(x)] dx = \frac{E[X^2]}{2\sigma^2} + \frac{1}{2} \ln(2\pi\sigma^2) = \frac{1}{2} + \frac{1}{2} \ln(2\pi\sigma^2) = \frac{1}{2} \ln(2\pi e\sigma^2)$$

O resultado depende **apenas da variância** — quanto maior a dispersão, maior a entropia diferencial. Este é o resultado clássico que mostra que, entre todas as distribuições contínuas com variância fixa, **a Normal é a que maximiza a entropia diferencial** (máxima incerteza compatível com uma dada variância) — resultado que será usado diretamente na dedução da capacidade de canal gaussiano (Capítulo 10).

Esse é um exemplar do **princípio da máxima entropia** (Jaynes, 1957): sob restrições conhecidas, a distribuição “mais honesta” (que não pressupõe nada além do que

se sabe) é a de máxima entropia. Cada restrição gera uma família: variância fixa $\rightarrow$ Normal; média fixa em $[0, \infty) \rightarrow$ Exponencial; suporte limitado $[a, b]$ sem outra informação $\rightarrow$ Uniforme. Repare que são exatamente as distribuições “onipresentes” do Capítulo 4 — a Teoria da Informação explica *por que* elas são onipresentes.

Propriedades da entropia diferencial (diferentes do caso discreto):

- $H(X + c) = H(X)$ — deslocamento não altera a entropia.
- $H(cX) = H(X) + \log |c|$ — escalonamento por c altera a entropia pelo log do valor absoluto de c .
- $H(CX) = H(X) + \log |\det C|$ para transformações matriciais (vetores).

Ou seja, ao contrário da entropia discreta, a entropia diferencial **não é invariante a mudanças de escala** — deve ser interpretada como medida relativa/comparativa, não como “quantidade absoluta de informação” no mesmo sentido do caso discreto. (Curiosidade útil: a *divergência KL* e a *informação mútua* contínuas **são** invariantes a mudanças invertíveis de coordenadas — por isso são preferidas quando se precisa de uma medida robusta no contínuo.)

8.2 Propriedades gerais (válidas nos dois casos)

1. $D(p\|g) \geq 0$ sempre.
2. $I(X, Y) \geq 0$, com igualdade sse independentes.
3. $H(X|Y) \leq H(X)$, com igualdade sse independentes (condicionar nunca aumenta, em média, a incerteza — “informação nunca faz mal”).

8.3 Outras definições de entropia

A entropia de Shannon é apenas um membro de uma família mais ampla de medidas de incerteza/diversidade:

Entropia de Rényi

Generaliza Shannon por um parâmetro de ordem α ($\alpha > 0$, $\alpha \neq 1$):

$$H_\alpha(X) = \frac{1}{1 - \alpha} \log \left(\sum_i p_i^\alpha \right)$$

Casos particulares importantes:

- $\alpha \rightarrow 0$: **Entropia de Hartley**, $H_0 = \log(N)$ — conta apenas o número de resultados possíveis, ignorando as probabilidades (é a medida de informação proposta por Hartley em 1928, vinte anos antes de Shannon).
- $\alpha \rightarrow 1$: recupera exatamente a **entropia de Shannon** (o limite se calcula com L'Hôpital).
- $\alpha = 2$: $H_2(X) = -\log \left(\sum_i p_i^2 \right)$ — a “entropia de colisão”, particularmente conveniente computacionalmente (ver estimador de Parzen, Capítulo 9); a quantidade $\sum_i p_i^2$ é a probabilidade de duas amostras independentes coincidirem.

- $\alpha \rightarrow \infty$: **Min-entropia**, $H_\infty = -\log(\sup_i p_i)$ — usada em criptografia como medida de “pior caso” de aleatoriedade de uma chave: um atacante ótimo chuta sempre o valor mais provável, e H_∞ mede a dificuldade *desse* chute, não do caso médio.

Vale sempre $H_0 \geq H_1 \geq H_2 \geq \dots \geq H_\infty$ (H_α é não-crescente em α). **Exemplo numérico** com $p = (0,5; 0,25; 0,25)$: $H_0 = \log_2 3 \approx 1,585$; $H_1 = 1,5$; $H_2 = -\log_2(0,375) \approx 1,415$; $H_\infty = 1$ bit. Valores pequenos de α dão mais peso a eventos raros (diversidade); valores grandes dão mais peso ao evento mais provável (dominância). Usada em ecologia (índices de diversidade — o índice de Simpson é essencialmente H_2), segurança (min-entropia) e física estatística.

Entropia de Boltzmann-Gibbs (origem física do nome “entropia”)

$$H = -k_B \sum_{\alpha} p_{\alpha} \log p_{\alpha}$$

onde k_B é a constante de Boltzmann e p_{α} é a probabilidade do sistema estar no microestado α . Matematicamente idêntica à entropia de Shannon (a menos da constante k_B) — foi, de fato, a inspiração histórica (por sugestão de John von Neumann) para Shannon batizar seu conceito de “entropia”. A anedota registrada é que von Neumann teria aconselhado: chame de entropia, “porque ninguém sabe o que a entropia realmente é, e num debate você sempre levará vantagem”. A conexão é profunda e não apenas nominal: o princípio de Landauer (1961) estabelece que apagar 1 bit de informação custa, no mínimo, $k_B T \ln 2$ de energia dissipada — informação é física.

Entropia de Tsallis

Generaliza Boltzmann-Gibbs por um parâmetro q :

$$H_q(p) = \frac{1}{q-1} \left(1 - \sum p^q \right)$$

Recupera Boltzmann-Gibbs quando $q \rightarrow 1$. Usada para sistemas com correlações de longo alcance (estatísticas não-extensivas): enquanto a entropia de Shannon/Gibbs é *aditiva* para sistemas independentes, a de Tsallis não é — e é exatamente essa não-aditividade que a torna adequada a sistemas fortemente correlacionados (proposta por Constantino Tsallis, físico do CBPF, Rio de Janeiro, em 1988).

Entropia de von Neumann (domínio quântico)

$$H(\rho) = -\text{tr}[\rho \log(\rho)]$$

onde ρ é a matriz densidade (Hermitiana, positiva, traço 1) que descreve um estado quântico. É o análogo direto da entropia de Shannon no domínio quântico: em vez de uma distribuição de probabilidade sobre estados clássicos, usam-se os autovalores de ρ como se fossem probabilidades. (Note a inversão histórica: von Neumann definiu

sua entropia em 1927 — *antes* de Shannon.) É a quantidade central da moderna informação quântica, medindo, por exemplo, o grau de emaranhamento de estados puros bipartidos.

Entropia Espectral

Aplica a fórmula de Shannon à **densidade espectral** de um sinal em vez de a uma distribuição de probabilidade:

$$H_{sp}(P) = - \sum_{i=f_l}^{f_h} P_i \log(P_i)$$

Aplicação notável: medir profundidade de sedação/anestesia via EEG — um espectro mais “espalhado” (maior entropia espectral) indica maior atividade/consciência; um espectro concentrado indica sedação profunda. Monitores comerciais de anestesia (módulo M-Entropy, GE Healthcare) implementam exatamente essa ideia na prática clínica.

Leituras complementares deste capítulo: COVER & THOMAS, *Elements of Information Theory* (cap. 8 — entropia diferencial; cap. 12 — máxima entropia); JAYNES, E. T. “Information Theory and Statistical Mechanics”, *Physical Review*, 1957; RÉNYI, A. “On Measures of Information and Entropy”, 1960; TSALLIS, C. “Possible Generalization of Boltzmann-Gibbs Statistics”, *J. Stat. Phys.*, 1988; LANDAUER, R. “Irreversibility and Heat Generation in the Computing Process”, *IBM Journal*, 1961; NIELSEN & CHUANG, *Quantum Computation and Quantum Information* (cap. 11, para a entropia de von Neumann).

Capítulo 9 — Codificação de Fonte e Compressão de Dados

9.1 O problema da codificação de fonte

Codificação de fonte: representar os símbolos de uma fonte de forma eficiente. Princípio universal: **símbolos mais frequentes recebem palavras-código menores; símbolos raros recebem palavras-código maiores** (o código Morse já usava essa ideia no século XIX: a letra mais frequente do inglês, ‘E’, é um único ponto ‘.’, enquanto ‘Q’, rara, é ‘- . - .’).

Formalizando: fonte com K símbolos, probabilidades p_k , palavras-código binárias de comprimento l_k . O comprimento médio:

$$L = \sum_k p_k \cdot l_k$$

A eficiência de codificação é $\eta = L_{min}/L$ (idealmente $\eta \rightarrow 1$).

9.2 O Primeiro Teorema de Shannon (Teorema da Codificação de Fonte)

$$L \geq H(A)$$

A entropia da fonte é o **limite fundamental** — não existe codificação sem perdas cujo comprimento médio fique abaixo da entropia. O limite é alcançável ($L_{min} = H(A)$) apenas em casos especiais (ver Kraft-McMillan abaixo). Combinando com a extensão de fonte do Capítulo 6 ($H(A^n) = nH(A)$), tem-se o resultado assintótico:

$$\lim_{n \rightarrow \infty} \frac{L_n}{n} = H(A)$$

— codificando blocos cada vez maiores de símbolos, a taxa de codificação por símbolo se aproxima arbitrariamente da entropia, ao custo de uma complexidade de decodificação crescente.

A ideia por trás da prova merece ser conhecida: pela **Propriedade de Equipartição Assintótica** (AEP — consequência da Lei dos Grandes Números aplicada a $-\frac{1}{n} \log p(X_1, \dots, X_n) \rightarrow H$), quando n é grande quase toda a probabilidade se concentra num conjunto de $\approx 2^{nH}$ *sequências típicas*, todas aproximadamente equiprováveis. Basta então enumerar as típicas com nH bits e ignorar (ou tratar à parte) as atípicas. É o mesmo mecanismo de concentração que fundamenta o teorema do canal (Capítulo 10).

9.3 Códigos prefixo e a Inequação de Kraft-McMillan

Um **código prefixo** é aquele em que nenhuma palavra-código é prefixo de outra. Isso garante que o código seja **unicamente decodificável** e, mais que isso, **instantâneo** (decodificável símbolo a símbolo, sem esperar bits futuros) — a decodificação corresponde a percorrer uma árvore binária até uma folha.

Contraexemplo para fixar: o código $\{a=0, b=01, c=011\}$ *não* é prefixo (0 é prefixo de 01). Recebendo 0 0 1 1..., o decodificador não sabe se começou com “a” ou se está no meio de um “b” ou “c” — precisa olhar adiante. Já $\{a=0, b=10, c=11\}$ é prefixo: cada bit lido desce um nível na árvore, e toda folha decide imediatamente.

Inequação de Kraft-McMillan: os comprimentos l_k de um código prefixo devem satisfazer

$$\sum_k 2^{-l_k} \leq 1$$

e, reciprocamente, se um conjunto de comprimentos satisfaz essa desigualdade, existe um código prefixo construível com esses comprimentos. A intuição geométrica: uma palavra de comprimento l “consome” a fração 2^{-l} das folhas da árvore binária infinita (todas as continuções possíveis ficam proibidas); o orçamento total é 1.

Limites do comprimento médio: para qualquer código prefixo de uma fonte com entropia $H(A)$:

$$H(A) \leq L < H(A) + 1$$

A igualdade à esquerda ocorre quando $p_k = 2^{-l_k}$ exatamente para todo k (código “absolutamente ótimo” — só possível quando todas as probabilidades são potências de $1/2$; é o que explica o “+1” residual do Huffman e a motivação da codificação por blocos e da codificação aritmética, que dilui esse resíduo).

9.4 Codificação de Huffman

Ideia: associar a cada símbolo uma palavra-código de comprimento aproximadamente igual à sua autoinformação $-\log_2 p_k$, aproximando o limite teórico da entropia. O algoritmo (David Huffman, 1952, então aluno de pós-graduação no MIT, resolvendo um

trabalho de curso!) é **guloso e provavelmente ótimo** entre todos os códigos símbolo a símbolo.

Algoritmo (passo a passo)

1. Liste os símbolos em ordem decrescente de probabilidade. Atribua os rótulos '0' e '1' aos dois símbolos de **menor** probabilidade.
2. Combine esses dois símbolos em um novo símbolo, com probabilidade igual à soma das duas. Reordene a lista.
3. Repita até restarem apenas dois símbolos.
4. Reconstrua cada palavra-código percorrendo o processo de trás para frente (retropropagação), concatenando os bits atribuídos em cada fusão pela qual o símbolo original passou.

Exemplo resolvido

Fonte com distribuição:

Símbolo	a_0	a_1	a_2	a_3	a_4
Probabilidade	0,4	0,2	0,2	0,1	0,1

Entropia da fonte: $H(A) = 2,12193$ bits/símbolo.

Uma construção possível de Huffman: fundir primeiro os dois menores ($a_3 + a_4 = 0,2$); reordenar $\{a_0:0,4, [a_1:0,2, a_2:0,2, a_3a_4:0,2]\}$; fundir dois dos 0,2 (ex.: $a_2 + a_3a_4 = 0,4$); reordenar $\{a_0:0,4, a_1:0,2, a_2a_3a_4:0,4\}$; fundir a_1 com $a_2a_3a_4$ ($= 0,6$); por fim fundir com a_0 . A árvore resultante:

Árvore de Huffman do exemplo resolvido (fusões da direita para a esquerda)

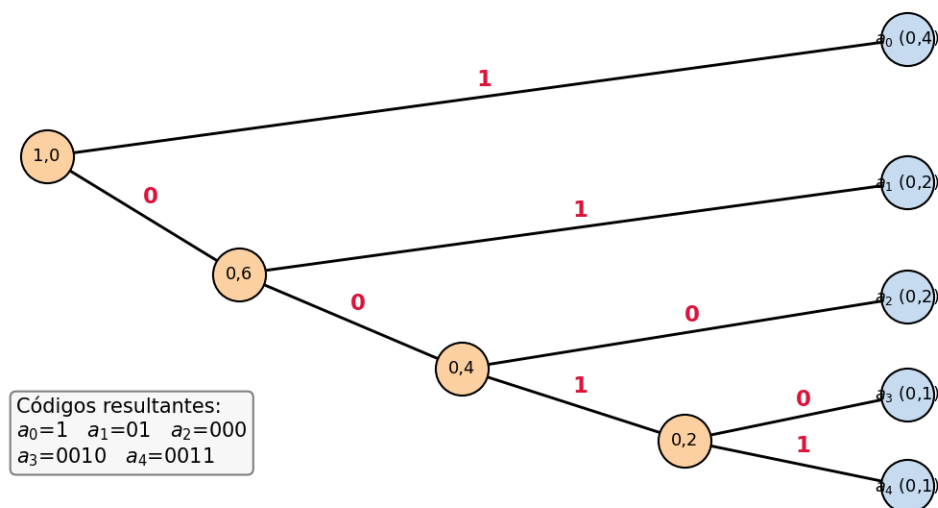


Figura 9: Árvore de Huffman do exemplo. Cada folha (azul) é um símbolo; cada nó interno (laranja) é uma fusão, com a probabilidade acumulada; o código de um símbolo é a sequência de bits da raiz até sua folha.

Símbolo	Código	l_k
a_0	1	1
a_1	01	2
a_2	000	3
a_3	0010	4
a_4	0011	4

$$L = 0,4(1) + 0,2(2) + 0,2(3) + 0,1(4) + 0,1(4) = 0,4 + 0,4 + 0,6 + 0,4 + 0,4 = 2,2$$

Confirma-se $H(A) \leq L < H(A) + 1$ ($2,122 \leq 2,2 < 3,122$), com L apenas 3,67% acima da entropia teórica. Verifique também Kraft: $2^{-1} + 2^{-2} + 2^{-3} + 2 \cdot 2^{-4} = 0,5 + 0,25 + 0,125 + 0,125 = 1$ — orçamento exatamente esgotado, como convém a um código ótimo. **Teste de decodificação:** a sequência 1 01 0010 1 decodifica sem ambiguidade como $a_0 a_1 a_3 a_0$ — nenhuma palavra é prefixo de outra.

Não-unicidade e limitações

O código de Huffman **não é único**: a atribuição de '0'/'1' em cada fusão é arbitrária, e empates de probabilidade podem ser resolvidos de formas diferentes. Diferentes execuções podem gerar códigos com o mesmo L ótimo, mas **variância diferente**:

$$\sigma^2 = \sum_k p_k (l_k - L)^2$$

Entre soluções igualmente ótimas em L , prefere-se a de menor variância (tamanhos de palavra mais previsíveis, úteis para buffers).

Limitação principal: exige conhecimento a priori das probabilidades exatas da fonte — nem sempre disponível ou estável (fontes dinâmicas exigiriam reconstruir a árvore continuamente). Além disso, por atribuir um número *inteiro* de bits por símbolo, desperdiça até quase 1 bit por símbolo em fontes muito enviesadas (ex.: $p = (0,99; 0,01)$ tem $H \approx 0,08$ bit, mas Huffman gasta 1 bit) — problema que a **codificação aritmética** (e a moderna ANS, usada no Zstandard do Facebook) resolve codificando a mensagem inteira como um único número em $[0, 1)$.

9.5 Codificação Lempel-Ziv-Welch (LZW)

Alternativa **adaptativa** (não exige estatísticas a priori) e de implementação mais simples que Huffman. Constrói um **dicionário dinâmico** de subsequências ainda não vistas. É a família de algoritmos (LZ77/LZ78/LZW — Lempel & Ziv 1977/78, refinamento de Welch 1984) por trás de GIF, TIFF, do antigo compress do Unix e, via LZ77+Huffman, do ZIP/PNG (algoritmo DEFLATE).

Algoritmo de compressão

1. Inicialize o dicionário com todas as strings de comprimento 1 (tipicamente ASCII).
2. Encontre a string mais longa W já no dicionário que casa com a entrada atual.
3. Emita o **índice** de W na saída; avance o ponteiro de leitura.
4. Adicione ao dicionário a string “ W + próximo símbolo”.
5. Repita até consumir toda a entrada.

Exemplo resolvido passo a passo

Sequência de entrada: 000101110010100101. Dicionário inicial: $\{0 \rightarrow 0, 1 \rightarrow 1\}$ (índices 0 e 1).

Passo	Casamento W	Próx. símbolo	Emita (índice)	Novo verbete
1	0	0	0	2: 00
2	0	1	0	3: 01
3	01	1	3	4: 011
4	1	0	1	5: 10
5	0	0	0	6: (já existe 00 → casa 00)*

Na prática, no passo 5 o maior casamento é 00 (índice 2), próximo símbolo 1, emite 2 e cria o verbete 6: 001 — e assim por diante: 010 (do verbete 3 + 0), 100, 101... Note o mecanismo essencial: **quanto mais repetitiva a entrada, mais longos os**

casamentos e menos índices são emitidos. Um resultado teórico notável (Ziv & Lempel, 1978) garante que, para fontes estacionárias ergódicas, a taxa do LZ converge assintoticamente para a entropia da fonte — o algoritmo “aprende” as estatísticas sozinho, sendo assim universal*.

Características: sem perdas; o dicionário **não precisa ser transmitido** junto com o arquivo (é reconstruído simetricamente durante a decodificação); usa tipicamente códigos de **tamanho fixo** (ex.: 12 bits) para índices, ao contrário do tamanho variável de Huffman; muito eficaz em dados com alta repetição de padrões (ex.: imagens GIF/TIFF).

9.6 Compressão com e sem perdas — visão geral

Tipo	Característica	Exemplos	Uso
Sem perdas (<i>lossless</i>)	Recupera os dados originais exatamente	Huffman, LZ77, LZ78, LZW	Texto, código-fonte, bancos de dados
Com perdas (<i>lossy</i>)	Descarta informação pouco perceptível	JPEG, MP3, MPEG	Imagem, áudio, vídeo

A compressão com perdas tem sua própria teoria — a **teoria taxa-distorção** (*rate-distortion*, também de Shannon, 1959), que quantifica o mínimo de bits necessário para representar uma fonte com distorção média no máximo D . O JPEG é um exemplo instrutivo do pipeline completo do curso: transformação para um domínio decorrelacionado (DCT — prima da PCA do Capítulo 15), quantização (a etapa com perdas, guiada pela percepção humana) e, por fim, codificação de entropia sem perdas (Huffman ou aritmética) nos coeficientes quantizados.

9.7 Estimando entropia a partir de dados reais

Na prática, a densidade de probabilidade $p_X(x)$ raramente é conhecida — precisa ser **estimada** a partir de amostras. Duas abordagens citadas no curso:

- **Expansão de Gram-Charlier/Edgeworth:** aproxima p_X como uma correção polinomial em torno de uma gaussiana, usando momentos/cumulantes de ordem superior. Funciona bem quando a densidade real está “próxima” de uma gaussiana; conecta-se ao conceito de **negentropia** ($NG(p_X) = H(p_G) - H(p_X)$, sempre ≥ 0 , zero se e só se p_X é gaussiana — uma medida de “quão não-gaussiana” é a distribuição, usada por exemplo em Análise de Componentes Independentes, ICA).
- **Estimador de Parzen (kernel density estimation):** aproxima qualquer densidade por uma soma de funções kernel centradas nas amostras, $p_X(x) = \frac{1}{N} \sum_i K(x - x_i, \sigma_i I)$. É especialmente conveniente para a entropia de Rényi de ordem 2, pois a integral de p^2 se resolve analiticamente em uma soma dupla

fechada sobre pares de amostras — a chamada “entropia quadrática de Rényi”, muito usada em *Information Theoretic Learning* (ITL).

Uma terceira via, elegante e prática, decorre da própria teoria deste capítulo: **comprimir para estimar**. Como nenhum compressor sem perdas fica abaixo da entropia, o tamanho do arquivo comprimido por um bom compressor universal (gzip, zstd) é um *limite superior empírico* de $n \cdot H$ — método simples e surpreendentemente eficaz para comparar a “aleatoriedade” de conjuntos de dados, e que antecipa a ponte compressão $\leftrightarrow$ complexidade do Capítulo 12.

Leituras complementares deste capítulo: HUFFMAN, D. “A Method for the Construction of Minimum-Redundancy Codes”, *Proc. IRE*, 1952; ZIV & LEMPEL, “A Universal Algorithm for Sequential Data Compression”, *IEEE Trans. IT*, 1977; WELCH, T. “A Technique for High-Performance Data Compression”, *IEEE Computer*, 1984; SAYOOD, K. *Introduction to Data Compression*, 5ª ed. (referência prática completa); SALOMON & MOTTA, *Handbook of Data Compression*; PRINCIPE, J. *Information Theoretic Learning*, Springer, 2010 (Parzen/Rényi em ML); MACKAY, caps. 4-6.

Capítulo 10 — Capacidade de Canal, Ruído e o Limite de Shannon

10.1 O problema da codificação de canal

Enquanto a codificação de fonte (Capítulo 9) busca **remover** redundância (compressão), a codificação de canal busca **inserir redundância controlada**, para proteger a mensagem contra os efeitos do ruído. (Reveja o diagrama de blocos de Shannon na abertura da Parte II: as duas operações são independentes e complementares — o chamado *princípio da separação*, também provado por Shannon: nada se perde, assintoticamente, ao projetar compressor e protetor de canal separadamente.)

Exemplo intuitivo: repetir cada bit três vezes ($0 \rightarrow 000$, $1 \rightarrow 111$) e, no receptor, decidir pelo bit majoritário — se apenas um dos três bits for corrompido, o erro é corrigido. Essa é a ideia por trás de qualquer código de canal, ainda que os esquemas práticos sejam mais sofisticados (Capítulo 11).

Quantificando o exemplo: com probabilidade de erro por bit $p = 0,1$, o código de repetição erra quando 2 ou 3 bits são corrompidos: $P_{\text{erro}} = 3p^2(1-p) + p^3 = 0,027 + 0,001 = 0,028$ — o erro caiu de 10% para 2,8%, mas a taxa despencou para $r = 1/3$. Repetindo 5, 7, 9... vezes, o erro cai mais, mas a taxa tende a zero. Antes de Shannon, acreditava-se que esse trade-off era inescapável: comunicação confiável exigiria taxa nula. O Segundo Teorema mostra que **não**: existe uma taxa positiva C abaixo da qual o erro pode ser tornado arbitrariamente pequeno.

Códigos de bloco: a mensagem é dividida em blocos de k bits, cada um mapeado em um bloco de $n > k$ bits (com $n - k$ bits de redundância). A taxa do código:

$$r = \frac{k}{n}$$

10.2 Segundo Teorema de Shannon (Teorema da Codificação do Canal)

Seja uma fonte com entropia $H(A)$ emitindo um símbolo a cada T_s segundos, e um canal discreto sem memória com capacidade C (bits por uso), usado a cada T_c

segundos.

Parte de existência: se

$$\frac{H(A)}{T_s} \leq \frac{C}{T_c}$$

existe um esquema de codificação tal que a informação pode ser transmitida e reconstruída com probabilidade de erro **arbitrariamente pequena**.

Parte de impossibilidade (recíproca): se a desigualdade for invertida, **não** é possível atingir erro arbitrariamente pequeno, qualquer que seja o esquema de codificação.

C/T_c é chamado de **taxa crítica** — o limite fundamental de transmissão confiável. É importante notar que este é um **teorema de existência não-construtivo**: Shannon prova (via um argumento probabilístico de “codificação aleatória”) que um bom código existe, mas não fornece o algoritmo para construí-lo — encontrar códigos práticos que se aproximem desse limite foi um desafio de décadas de pesquisa em engenharia (culminando, décadas depois, em códigos como Turbo Codes e LDPC).

A ideia da prova é uma das mais belas da matemática aplicada: escolha 2^{nr} palavras-código *ao acaso*; pela AEP, cada saída recebida é “compatível” com $\approx 2^{nH(X|Y)}$ entradas típicas, e se $r < C = \max I(X, Y)$ a chance de duas palavras-código colidirem na mesma região de compatibilidade vai a zero quando $n \rightarrow \infty$. Ou seja: *um código aleatório típico é bom* — o difícil é achar um que também seja decodificável com baixo custo computacional.

Caso especial — Canal Binário Simétrico (BSC): definindo a taxa de codificação $r = T_c/T_s$, a condição de confiabilidade se reduz a $r \leq C$.

10.3 Capacidade de canal: definição geral

$$C = \max_{p_X(x)} I(X, Y)$$

A capacidade é o **máximo da informação mútua** entre entrada e saída, otimizado sobre todas as distribuições de entrada possíveis. Depende apenas das probabilidades condicionais $P(y|x)$ que caracterizam o canal (não da entrada escolhida em si — é uma propriedade *do canal*).

Propriedades: $C \geq 0$; $C \leq \log |X|$ e $C \leq \log |Y|$; $I(X, Y)$ é contínua e **côncava** em $p_X(x)$ — o que garante que o máximo existe e é bem definido (problema de otimização côncava, sem múltiplos ótimos locais; na prática, resolve-se numericamente com o algoritmo de Blahut-Arimoto, 1972).

Canal Binário Simétrico (BSC)

Com probabilidade de erro (inversão de bit) p :

$$C = 1 - H(p)$$

onde $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ é a entropia binária. (O leitor que resolveu o exemplo numérico da seção 7.5 já *derivou* essa fórmula para $p = 0,1$: $C = 1 - H_b(0,1) \approx 0,531$ bit/uso.)

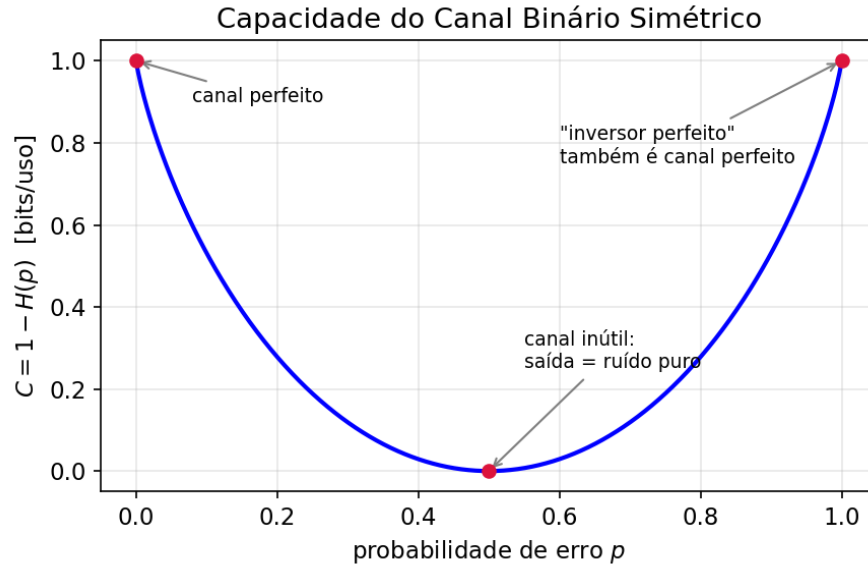


Figura 10: Capacidade do BSC em função da probabilidade de erro. A simetria em torno de $p = 0,5$ revela algo curioso: um canal que inverte quase todos os bits é quase perfeito — basta o receptor inverter de volta.

A curva $C(p)$ é simétrica em torno de $p = 0,5$: quando $p = 0$ (sem ruído), $C = 1$ bit/uso (máximo); quando $p = 0,5$, $C = 0$ — o canal é “inútil” (a saída não carrega nenhuma informação sobre a entrada, pois é indistinguível de ruído puro).

Canal com Apagamento (BEC) e canal combinado

Em um **canal binário com apagamento (BEC)**, o receptor pode receber 0, 1, ou um símbolo de “apagamento” — o bit foi perdido, mas o receptor *sabe* que perdeu, sem confundir com outro valor. É o modelo natural da internet: um pacote TCP/IP que não chega é detectado (por numeração de sequência), não confundido com outro.

Para um canal que combina apagamento (probabilidade ϵ) com erro de inversão (probabilidade p , condicional a não ter sido apagado):

$$C = (1 - \epsilon)(1 - H(p))$$

Interpretação: $(1 - \epsilon)$ é a fração de bits que chegam (não apagados); $(1 - H(p))$ é a capacidade residual tipo BSC sobre os bits que chegaram. Casos particulares: $\epsilon = 0$ recai no BSC puro; $p = 0$ recai no BEC puro ($C = 1 - \epsilon$). Note a diferença qualitativa:

com $\epsilon = 0,1$ e $p = 0$, $C = 0,9$; com $p = 0,1$ e $\epsilon = 0$, $C \approx 0,531$. **Saber onde está o dano é muito mais barato do que não saber** — apagamento custa ϵ ; inversão custa $H(p) \gg p$ para p pequeno.

10.4 Terceiro Teorema de Shannon: capacidade do canal gaussiano (Shannon-Hartley)

Este é o resultado mais famoso de toda a Teoria da Informação. Considere um canal contínuo de largura de banda B Hz, com ruído aditivo gaussiano branco (AWGN) de densidade espectral $N_0/2$, e potência média de transmissão P .

Derivação (ideia central, passo a passo):

1. A saída é $Y_k = X_k + N_k$, com X_k e N_k independentes. A informação mútua é $I(X_k, Y_k) = H(Y_k) - H(Y_k|X_k) = H(Y_k) - H(N_k)$.
2. Como $H(N_k)$ não depende de como escolhemos codificar X_k , **maximizar I equivale a maximizar $H(Y_k)$** .
3. Para uma variância fixa, a entropia diferencial é máxima quando a distribuição é **gaussiana** (resultado do Capítulo 8) — logo, a entrada ótima X_k também deve ser gaussiana.
4. Com $Y_k \sim N(0, P + \sigma^2)$ e $N_k \sim N(0, \sigma^2)$ (onde $\sigma^2 = N_0 B$ é a potência do ruído), usando a fórmula de entropia diferencial gaussiana do Capítulo 8:

$$C = \frac{1}{2} \log_2 \left(1 + \frac{P}{\sigma^2} \right) \text{ bits por transmissão}$$

(De fato: $C = \frac{1}{2} \ln[2\pi e(P + \sigma^2)] - \frac{1}{2} \ln[2\pi e \sigma^2] = \frac{1}{2} \ln \frac{P + \sigma^2}{\sigma^2}$ — as constantes $2\pi e$ cancelam.)

5. Como o canal é amostrado $2B$ vezes por segundo (teorema de Nyquist), a capacidade por unidade de tempo é:

$$C = B \log_2 \left(1 + \frac{P}{N_0 B} \right) \text{ bps}$$

Reescrevendo com a razão sinal-ruído $SNR = P/(N_0 B)$:

$$C = B \log_2(1 + SNR)$$

Leitura da fórmula

- C cresce apenas **logaritmicamente** com o SNR — dobrar a potência de transmissão não dobra a capacidade.
- C cresce **linearmente** com a largura de banda B (mas, atenção: aumentar B também aumenta o ruído total $N_0 B$ recebido, então o ganho não é ilimitado; no limite $B \rightarrow \infty$, $C \rightarrow \frac{P}{N_0} \log_2 e$ — finito!).

- É um **limite superior absoluto**: nenhum esquema de codificação, por mais sofisticado, consegue transmitir de forma confiável a uma taxa acima de C . Para se aproximar desse limite, o sinal transmitido deve ter estatísticas próximas às de ruído gaussiano branco — ou seja, o “**código ótimo**” **tende a parecer ruído** (dispersão máxima de energia, sem padrões redundantes exploráveis). Há aqui uma simetria elegante com o Capítulo 9: dados perfeitamente *comprimidos* também parecem ruído — no ponto ótimo, tanto a fonte codificada quanto o sinal de canal ideal são indistinguíveis de aleatoriedade pura.

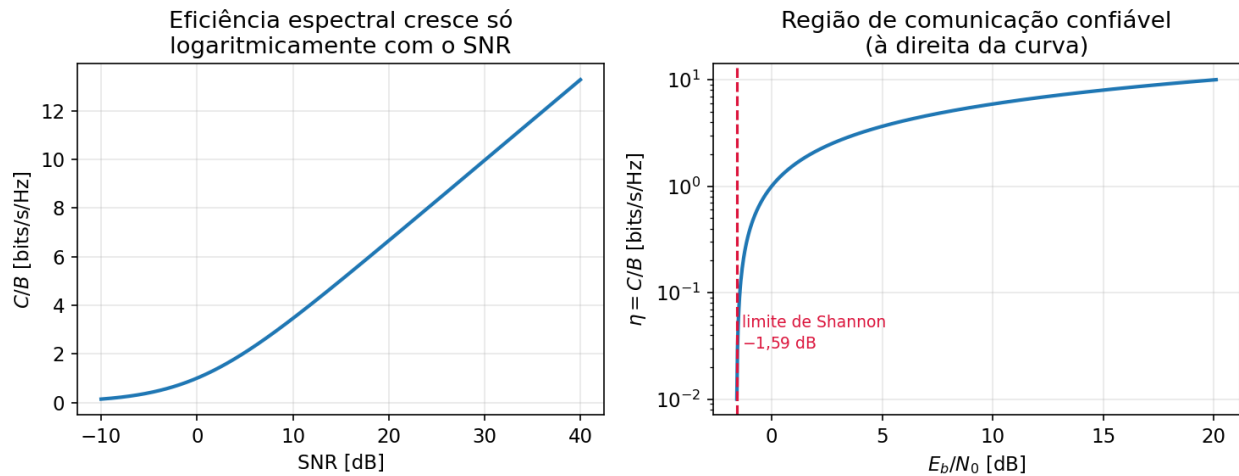


Figura 11: À esquerda: a eficiência espectral C/B cresce apenas logarithmicamente com o SNR. À direita: a fronteira de comunicação confiável no plano $E_b/N_0 \times \eta$, com o limite assintótico de $-1,59$ dB.

Exemplo numérico resolvido

Canal com $B = 5$ MHz e $SNR = 100$:

$$C = 5.000.000 \times \log_2(101) \approx 5.000.000 \times 6,658 \approx 33,29 \text{ Mbps}$$

Segundo exemplo (histórico — o modem discado): linha telefônica com $B \approx 3,1$ kHz e $SNR \approx 30$ dB ($= 1000$): $C = 3100 \times \log_2(1001) \approx 3100 \times 9,97 \approx 30,9$ kbps. Eis por que os modems dos anos 1990 estacionaram em 28,8/33,6 kbps: **estavam encostados no limite de Shannon da linha** (o modem “56k” só ultrapassava isso por operar num enlace parcialmente digital, com menos ruído de quantização).

Eficiência espectral e o limite de Shannon em E_b/N_0

Definindo E_b = energia por bit (com $P = E_b C$ em um “sistema ideal” onde a taxa de transmissão R_b iguala C) e a **eficiência espectral** $\eta = C/B$ (bits/s por Hz), pode-se reescrever a fórmula em termos de E_b/N_0 :

$$\frac{E_b}{N_0} = \frac{2^\eta - 1}{\eta}$$

Quando $\eta \rightarrow 0$ (banda muito larga em relação à taxa), esse limite converge para o famoso valor assintótico de $E_b/N_0 = \ln 2 \approx 0,693$, ou $\approx -1,59$ **dB** — abaixo do qual nenhuma comunicação confiável é possível, não importa quão sofisticada seja a codificação. Tecnologias modernas (4G/5G) usam técnicas como OFDM e MIMO precisamente para se aproximar desse limite teórico; as sondas de espaço profundo (Voyager, New Horizons), operando com potências minúsculas a bilhões de km, vivem no regime de η baixíssimo, o mais próximo possível dos $-1,59$ dB.

Leituras complementares deste capítulo: SHANNON, 1948 (Parte II do artigo); COVER & THOMAS (caps. 7 e 9); HAYKIN, S. *Sistemas de Comunicação*, 5ª ed. (tratamento de engenharia, em português); PROAKIS & SALEHI, *Digital Communications*; SKLAR, B. *Digital Communications: Fundamentals and Applications* (cap. sobre o “Shannon limit” e a interpretação de E_b/N_0); BLAHUT, R. “Computation of Channel Capacity and Rate-Distortion Functions”, *IEEE Trans. IT*, 1972.

Capítulo 11 — Códigos Corretores de Erro

11.1 Motivação

Mesmo com um canal de alta capacidade, erros são inevitáveis (atenuação, interferência eletromagnética, imperfeições de hardware). A ideia central dos códigos corretores é **sacrificar eficiência de transmissão em troca de confiabilidade**, inserindo redundância controlada que permite ao receptor detectar e, em alguns casos, corrigir erros.

O conceito geométrico que unifica todo o capítulo é a **distância de Hamming** d entre duas palavras (número de bits em que diferem): um código cuja menor distância entre palavras-código válidas é d_{min} consegue **detectar** até $d_{min} - 1$ erros e **corrigir** até $\lfloor (d_{min} - 1)/2 \rfloor$ erros — cada palavra válida é o centro de uma “esfera” de raio de correção, e as esferas não podem se sobrepor. Projetar bons códigos é, geometricamente, o problema de *empacotar o maior número possível de esferas* no hipercubo $\{0, 1\}^n$.

11.2 Categorias de códigos

- **Códigos de Hamming:** um dos exemplos mais antigos (Richard Hamming, anos 1950 — motivado, conta a lenda, pela frustração de ver o computador do Bell Labs abortar seus jobs de fim de semana por erros de leitura), projetados para corrigir automaticamente erros de bit único, adicionando bits de verificação que satisfazem relações matemáticas específicas.
- **Códigos Cíclicos:** a sequência codificada permanece válida mesmo após deslocamentos cíclicos — úteis para fluxos contínuos (TV digital, armazenamento).
- **Códigos Convolucionais:** geram redundância envolvendo a sequência de dados com sequências geradoras fixas — adequados para comunicação sem fio e espaço profundo (decodificados eficientemente pelo algoritmo de Viterbi, 1967).
- **Turbo Codes e LDPC (Low-Density Parity-Check):** códigos modernos de alto desempenho, usados em comunicação por satélite e redes sem fio avançadas, que se aproximam do limite de Shannon. Nota histórica saborosa: os Turbo Codes (Berrou et al., 1993) chegaram a $\sim 0,5$ dB do limite e foram recebidos com incredulidade; os LDPC haviam sido inventados por **Gallager em 1962** e ficaram esquecidos por 30 anos — o hardware da época não conseguia executá-los. Hoje LDPC está no Wi-Fi 6, no 5G e nos SSDs.

11.3 Técnicas de detecção e correção

Técnica	O que faz	Detecta?	Corrige?
Paridade	1 bit extra torna o total par/ímpar	Erros ímpares apenas	Não
Checksum	Soma das palavras de dados	Mais tipos de erro que paridade	Não
CRC	Trata os dados como polinômio, divide por polinômio gerador	Padrões de erro complexos	Não (por si só)
FEC (<i>Forward Error Correction</i>)	Redundância suficiente para correção sem retransmissão	Sim	Sim

Exemplo resolvido completo — Hamming(7,4)

O código de Hamming(7,4) transmite $k = 4$ bits de dados ($d_1 d_2 d_3 d_4$) em $n = 7$ bits, acrescentando 3 bits de paridade:

$$p_1 = d_1 \oplus d_2 \oplus d_4, \quad p_2 = d_1 \oplus d_3 \oplus d_4, \quad p_3 = d_2 \oplus d_3 \oplus d_4$$

com a palavra transmitida na ordem ($p_1 p_2 d_1 p_3 d_2 d_3 d_4$) — cada bit de paridade nas posições que são potências de 2.

Codificação: para os dados 1011: $p_1 = 1 \oplus 0 \oplus 1 = 0$; $p_2 = 1 \oplus 1 \oplus 1 = 1$; $p_3 = 0 \oplus 1 \oplus 1 = 0$. Palavra transmitida: **0 1 1 0 0 1 1**.

Canal corrompe o 5º bit: recebe-se 0 1 1 0 **1** 1 1.

Decodificação (síndrome): o receptor recomputa as três paridades sobre o que recebeu; cada uma “vigia” um subconjunto de posições: s_1 (posições 1,3,5,7) = $0 \oplus 1 \oplus 1 \oplus 1 = 1$; s_2 (2,3,6,7) = $1 \oplus 1 \oplus 1 \oplus 1 = 0$; s_3 (4,5,6,7) = $0 \oplus 1 \oplus 1 \oplus 1 = 1$. A síndrome ($s_3 s_2 s_1$) = $101_2 = 5$ **aponta diretamente a posição errada:** bit 5. Inverte-se o bit 5 e recuperam-se os dados 1011 intactos. Elegância pura: a síndrome é o *endereço binário do erro* ($d_{min} = 3$: corrige 1 erro, detecta 2).

11.4 Taxa de código e o trade-off fundamental

$$R = \frac{k}{n}$$

(bits de dados sobre bits totais após codificação). Taxa mais alta = menos redundância = maior eficiência de transmissão, mas menor capacidade de correção. É o mesmo trade-off central visto no Capítulo 10 ($r \leq C$): quanto mais se protege contra erro,

menos “espaço útil” sobra no canal. No Hamming(7,4) do exemplo, $R = 4/7 \approx 0,57$ — comprou-se a correção de qualquer erro simples ao preço de 43% de overhead.

Exemplos de aplicação do trade-off: comunicação no espaço profundo (atraso enorme, retransmissão inviável) usa taxa de código baixa com forte correção; streaming de vídeo ao vivo (tolerância a pequenas perdas, alta demanda de tempo real) usa taxa alta com correção mais leve.

11.5 Aplicações modernas

- **Comunicação por satélite:** Turbo Codes e LDPC.
- **TV digital:** Códigos Cíclicos.
- **Armazenamento (HD/SSD):** Códigos de Hamming e Reed-Solomon.
- **Wi-Fi:** Códigos Convolucionais e Turbo Codes (e LDPC a partir do Wi-Fi 6/802.11ax).
- **Cotidiano invisível:** o QR Code usa Reed-Solomon — é por isso que continua legível mesmo com até ~30% da imagem danificada ou coberta por um logotipo; CDs e DVDs usam Reed-Solomon intercalado (um arranhão radial vira muitos erros pequenos e corrigíveis, em vez de um bloco perdido); a memória ECC de servidores usa variantes de Hamming (SECDED: corrige 1, detecta 2); o 5G adotou **códigos polares** (Arikan, 2009) — a primeira família *provadamente* capaz de atingir a capacidade de canais simétricos.

Leituras complementares deste capítulo: HAMMING, R. “Error Detecting and Error Correcting Codes”, *Bell System Technical Journal*, 1950; LIN & COSTELLO, *Error Control Coding*, 2ª ed. (a referência da área); MOON, T. *Error Correction Coding: Mathematical Methods and Algorithms*; BERROU, GLAVIEUX & THITIMAJSHIMA, “Near Shannon Limit Error-Correcting Coding: Turbo-Codes”, *ICC*, 1993; GALLAGER, R. *Low-Density Parity-Check Codes*, MIT Press, 1963; ARIKAN, E. “Channel Polarization”, *IEEE Trans. IT*, 2009.

Capítulo 12 — Complexidade de Kolmogorov

12.1 Uma segunda forma de medir informação

Shannon mede informação através de **probabilidade**: a informação de um evento x é $-\log[p_X(x)]$. Em 1965, o matemático russo Andrei Kolmogorov (o mesmo dos axiomas do Capítulo 2 — e, de forma independente, Ray Solomonoff em 1964 e Gregory Chaitin em 1966) propôs uma definição alternativa, **sem qualquer referência a probabilidades**: a complexidade algorítmica (ou descritiva) de um objeto é o comprimento do **menor programa** capaz de gerá-lo.

“A Complexidade de Kolmogorov mede a quantidade de informação de um objeto pelo tamanho do menor programa (algoritmo) capaz de produzi-lo.”

Exemplo canônico: compare duas sequências de 20 bits:

- 11111111111111111111 — pode ser gerada por um programa curto: FOR I=1 TO 20 PRINT 1. **Baixa complexidade** (string “regular”/compressível).
- 10110011010111011010 — não tem padrão aparente; o programa mais curto conhecido é essencialmente PRINT 10110011010111011010, quase do tamanho da própria string. **Alta complexidade** (string “aleatória”/incompressível).

Exemplo mais dramático: o primeiro bilhão de dígitos de π *parece* perfeitamente aleatório (passa em todos os testes estatísticos usuais), mas tem Complexidade de Kolmogorov minúscula: um programa de poucas linhas (uma série convergente, ex.: fórmula de Chudnovsky) gera quantos dígitos se queira. π é a demonstração viva de que “parecer aleatório” e “ser incompressível” são coisas diferentes — e de que a complexidade mede a segunda.

Essa ideia conecta-se com a **Navalha de Occam** (Guilherme de Ockham, 1287-1347): entre explicações igualmente válidas, prefere-se a mais simples — e “mais simples” pode ser formalizado precisamente como “de menor complexidade de Kolmogorov”. (Solomonoff levou isso às últimas consequências: sua *inferência indutiva universal* pesa cada hipótese por $2^{-K(\text{hipótese})}$ — a formalização matemática da navalha, e um dos fundamentos teóricos do aprendizado de máquina; o princípio prático **MDL** — *Minimum Description Length*, Rissanen 1978 — é sua versão computável: o melhor modelo é o que minimiza [tamanho do modelo] + [tamanho dos dados codificados pelo modelo], um antídoto formal contra o overfitting do Capítulo 14.)

12.2 Definição formal

Seja U um computador (interpretador) universal, p um programa e $U(p)$ a saída de U ao executar p . A **Complexidade de Kolmogorov** de uma string x é:

$$K_U(x) = \min_{p: U(p)=x} l(p)$$

isto é, o comprimento do menor programa que produz x como saída.

Teorema da Invariância (existência de função ótima)

A definição parece depender da escolha do interpretador U — mas essa dependência é limitada: existe uma função U (“ótima” ou “universal”) tal que, para qualquer outro interpretador A , existe uma constante c_A (**independente de x !**) tal que

$$K_U(x) \leq K_A(x) + c_A$$

Prova (ideia): dada a descrição mais curta t de x relativa a A (ou seja, $A(t) = x$), o interpretador universal U pode *simular* A : basta um programa (p, t) onde p é um “interpretador de A ” escrito na linguagem de U (tamanho fixo $|p|$) seguido da descrição t . Logo $K_U(x) \leq |p| + |t| = c_A + K_A(x)$. ■

Consequência: todas as funções ótimas dão essencialmente a mesma complexidade, **a menos de uma constante aditiva** — por isso a notação $K(x)$ (sem subscrito) é usada por convenção, entendendo-se que há uma constante de ambiguidade fixa, independente de x . Em termos práticos: medir a complexidade “em Python” ou “em C” difere no máximo pelo tamanho de um interpretador de uma linguagem escrito na outra — irrelevante para strings grandes.

Um limite superior trivial: $K(x) \leq |x| + C$ — basta usar a própria string x como sua descrição (o programa “imprima literalmente isto”).

Advertência fundamental (não-computabilidade): $K(x)$ **não é computável** — não existe algoritmo que, dado x , retorne $K(x)$. (Esboço: se existisse, poderíamos procurar “a primeira string com $K > n$ ” — que seria descrita pelo próprio procedimento de busca usando $\approx \log n$ bits, contradição; é o mesmo argumento de Berry/Gödel da seção 12.4.) Na prática, usa-se compressores reais como aproximações superiores de K — a base da métrica NCD (*Normalized Compression Distance*, Cilibrasi & Vitányi, 2005), que agrupa textos, genomas e músicas por similaridade “sem saber nada” sobre o domínio: comprime-se x , y e a concatenação xy e compara-se.

12.3 Strings incompressíveis

Proposição: a fração de strings de comprimento n com $K(x) < n - k$ não excede 2^{-k} .

Prova: há 2^n strings de comprimento n , mas apenas menos de 2^{n-k} descrições potenciais de comprimento $< n - k$. ■

Consequência direta: existem strings incompressíveis (com $K(x) \geq n$) — há 2^n strings de comprimento n , mas no máximo $2^n - 1$ descrições de comprimento $< n$, logo pelo menos uma string não admite descrição mais curta que si mesma. E mais: pela proposição, a *esmagadora maioria* das strings é quase incompressível — apenas 1 em 1024 pode ser comprimida em 10 bits ou mais ($k = 10$).

Curiosidade paradoxal: não é possível encontrar *efetivamente* (por algoritmo) uma string incompressível de comprimento n dado. Se fosse possível, o próprio algoritmo (mais o valor de n , que precisa de $\approx \log n$ bits) seria uma descrição de tamanho $\log n + c$ — e para n grande, $\log n + c < n$, contradizendo a incompressibilidade. Esse mesmo argumento de contagem mostra por que **nenhum algoritmo de compressão pode comprimir todo arquivo possível**: não há espaço suficiente de descrições curtas para todas as entradas. (Corolário anti-marketing: qualquer produto que prometa “comprimir qualquer arquivo em pelo menos 1 bit” é matematicamente fraudulento — se comprimissem todos, dois arquivos distintos colidiriam na mesma saída.)

12.4 O Teorema de Gödel via Complexidade de Kolmogorov

Uma das aplicações mais espetaculares desta teoria: uma prova do **Teorema da Incompletude de Gödel** em poucas linhas (formulação sugerida por Gregory Chaitin).

Teorema: existe um número m tal que, para todo x , a proposição “ $K(x) \geq m$ ” não é demonstrável — mesmo sendo verdadeira para infinitos valores de x (já que só há finitas descrições de comprimento $< m$).

Prova (por absurdo): suponha que, para todo m , exista algum x tal que “ $K(x) \geq m$ ” seja demonstrável. Um algoritmo hipotético, dado m , poderia enumerar todos os teoremas de um sistema formal até encontrar um da forma “ $K(x) \geq m$ ” e retornar x . Isso faria de m uma descrição de x (via esse algoritmo fixo), logo $K(x) \leq \log m + C$. Mas por hipótese “ $K(x) \geq m$ ” é verdadeiro (todo teorema demonstrado é verdadeiro) — logo $m \leq \log m + C$, desigualdade falsa para m grande. Contradição. ■

A leitura filosófica de Chaitin: um sistema formal com “ n bits de axiomas” não consegue provar que objetos têm complexidade muito maior que n — **não se extrai de um sistema mais informação do que se colocou nele**. A incompletude deixa de parecer uma patologia lógica e passa a ser um fato de *contabilidade informacional*.

Esse argumento formaliza o **paradoxo de Berry** (Bertrand Russell, 1908): “o menor inteiro que precisa de mais de mil palavras para ser descrito” — uma frase que parece descrever esse número usando poucas palavras, gerando um paradoxo aparente. A resolução está em distinguir “menor elemento com a propriedade P ” (bem definido) de “descrição” (que só a Complexidade de Kolmogorov formaliza rigorosamente): a versão corrigível é “o primeiro inteiro tal que ‘ $K(N) > m$ ’ é demonstrável” — o mesmo argumento do Teorema de Gödel.

12.5 Aleatoriedade formalizada sem probabilidade

Pergunta motivadora: por que uma placa de carro “7777 ZZ 77” parece mais “extraordinária” que “7353 NY 42”, se ambas têm exatamente a mesma probabilidade sob um modelo uniforme?

A resposta intuitiva: a primeira pertence a um **conjunto pequeno e facilmente descritível** (“dígitos repetidos”), enquanto a segunda não tem nenhuma propriedade especial que a coloque em um conjunto pequeno — ela é “apenas um número”. Essa intuição é exatamente o que a Complexidade de Kolmogorov formaliza.

Definição (sequências finitas): uma string é “aleatória” na medida em que sua complexidade se aproxima do seu próprio comprimento — a melhor forma de descrevê-la é simplesmente escrevê-la por extenso.

Definição (sequências infinitas): requer cuidado técnico adicional (a noção de **complexidade prefixa**, que evita a necessidade de codificar separadamente o comprimento da descrição). Com essa correção, x é dita aleatória se $\exists C \forall n : K(x_{1:n}) > n - C$.

Equivalência de Levin-Schnorr (anos 1970): a incompressibilidade de uma sequência infinita e sua resistência a **todo teste algorítmico de aleatoriedade** (no sentido de Martin-Löf) são equivalentes — um dos resultados centrais da Teoria Algorítmica da Informação, que permite reformular teoremas clássicos de probabilidade (“para quase todo x , vale P ”) como afirmações sobre sequências aleatórias *individuais* (“para toda sequência aleatória, vale P ”). Note o ganho conceitual: a probabilidade clássica só fala de *conjuntos* de medida 1 e jamais de uma sequência específica; a teoria algorítmica finalmente dá sentido rigoroso à frase “esta sequência é aleatória”.

Curiosidade: uma sequência infinita x é computável (recursiva) se e somente se $\exists C \forall n : K(x_{1:n} \mid n) < C$ (Teorema de Meyer) — as sequências mais “simples” são exatamente as computáveis. No extremo oposto vive a constante Ω de Chaitin (a probabilidade de parada de um computador universal): um número real perfeitamente definido cujos bits formam uma sequência Martin-Löf-aleatória — conhecê-los resolveria o problema da parada; é o objeto matemático “maximamente incognoscível”.

12.6 Complexidade de Kolmogorov como precursor da entropia

Fechando a conexão com o restante do curso: para uma variável aleatória X , Shannon define a informação de um evento $X = x$ como $-\log[p_X(x)]$; a complexidade descritiva correspondente é $\lceil -\log[p_X(x)] \rceil$ — o comprimento necessário para codificar x usando um código de Shannon ótimo.

A Complexidade de Kolmogorov **generaliza** essa ideia sem depender de uma distribuição de probabilidade conhecida a priori: mede a informação pelo tamanho do menor programa, não pela probabilidade do evento. Notavelmente, para fontes i.i.d., o valor esperado da Complexidade de Kolmogorov por símbolo converge para a **entropia de Shannon**: $\frac{1}{n}E[K(X_1 \dots X_n)] \rightarrow H(X)$ — as duas teorias, construídas

sobre fundações completamente diferentes (probabilidade $\times$ computabilidade), concordam no caso médio. Por isso ela é considerada um fundamento alternativo (mais geral, e não-probabilístico) do próprio conceito de informação. O quadro completo das três medidas do curso: **Shannon** mede a informação *média de uma fonte*; **Kolmogorov** mede a informação *de um objeto individual*; e a compressão prática (Capítulo 9) é a ponte operacional entre as duas.

Leituras complementares deste capítulo: DURAND & ZVONKIN, “Kolmogorov Complexity” (capítulo-base do curso); LI & VITÁNYI, *An Introduction to Kolmogorov Complexity and Its Applications*, 4ª ed., Springer (a bíblia da área); CHAITIN, G. *Meta Math! The Quest for Omega*, 2005 (divulgação pelo próprio autor); KOLMOGOROV, “Three Approaches to the Definition of the Concept of ‘Quantity of Information’”, 1965; SOLOMONOFF, “A Formal Theory of Inductive Inference”, 1964; CILIBRASI & VITÁNYI, “Clustering by Compression”, *IEEE Trans. IT*, 2005; GRÜNWALD, P. *The Minimum Description Length Principle*, MIT Press, 2007; HOFSTADTER, D. *Gödel, Escher, Bach*, 1979.

PARTE III – BIG DATA, APRENDIZADO DE MÁQUINA E RECONHECIMENTO DE PADRÕES

Capítulo 13 — Big Data: Frameworks e Processamento Distribuído

13.1 Por que processamento distribuído?

Quando o volume de dados excede a capacidade de uma única máquina, é preciso distribuir o processamento entre vários nós de um **cluster**. O framework clássico para isso é o **MapReduce**.

A mudança de paradigma merece destaque: em vez de *levar os dados até o programa* (o gargalo é a rede — mover petabytes é inviável), o MapReduce *leva o programa até os dados* — cada nó processa o pedaço que já está em seu disco local (princípio da *data locality*, apoiado em sistemas de arquivos distribuídos como o HDFS, que replica cada bloco em ~ 3 nós). Isso também resolve a **tolerância a falhas**: num cluster de milhares de máquinas baratas, falhas são a regra, não a exceção; se um nó cai, o framework simplesmente reexecuta suas tarefas em outra réplica dos dados, sem intervenção do programador.

13.2 MapReduce

Ideia central: o programador define apenas duas funções — **Map** e **Reduce** — e o framework cuida de toda a complexidade de paralelização, particionamento e comunicação entre nós. Os dados fluem como tuplas (chave, valor).

$$\text{map} : (k_1, v_1) \rightarrow \text{list}(k_2, v_2), \quad \text{reduce} : (k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$$

Desenvolvido pelo Google (Dean & Ghemawat, 2004); a implementação open-source mais popular é o **Apache Hadoop**. Entre o Map e o Reduce existe uma fase implícita — o *shuffle* — em que o framework agrupa, através da rede, todos os pares de mesma chave no mesmo redutor; é tipicamente a fase mais cara do job (ver seção de custo).

Exemplo canônico: WordCount

Contar a ocorrência de cada palavra em um conjunto de textos:

```
map(line_number, text):
    for each word in word_list:
        emit(word, 1)
```

```
reduce(word, values[]):
    word_count = sum(values)
    emit(word, word_count)
```

O Map emite (palavra, 1) para cada ocorrência; o Reduce soma todos os valores associados à mesma chave — essa é, em essência, a operação de **Agregação** da álgebra relacional.

Rastreamento numérico completo para as linhas "o dado vira informação" e "o dado vira conhecimento":

- *Map (linha 1):* (o,1), (dado,1), (vira,1), (informação,1); *Map (linha 2):* (o,1), (dado,1), (vira,1), (conhecimento,1) — os dois maps rodam **em paralelo**, em nós diferentes.
- *Shuffle:* agrupa por chave $\rightarrow$ (o,[1,1]), (dado,[1,1]), (vira,[1,1]), (informação,[1]), (conhecimento,[1]).
- *Reduce:* (o,2), (dado,2), (vira,2), (informação,1), (conhecimento,1).

O mesmo esqueleto conta acessos por URL em logs de servidores, calcula o histograma de símbolos de um arquivo gigante (exatamente o insumo do Huffman do Capítulo 9!) ou soma vendas por região.

Operações da álgebra relacional via MapReduce

Operação	Map	Reduce
Seleção (σ_C)	Testa condição C ; emite (t, t) se verdadeira	Identidade
Projeção (π_S)	Remove atributos fora de S ; emite (t', t')	Remove duplicatas
União	Emite (t, t) de ambas as relações	Remove duplicatas
Interseção	Emite (t, t) de ambas	Emite só se presente nas duas
Diferença ($P - S$)	Marca origem (t, P) ou (t, S)	Emite se só está em P
Junção natural ($P \bowtie S$)	Emite $(b, P(a))$ e $(b, S(c))$ pela chave comum b	Combina pares (a, c) por chave
Agrupamento/Agregação	Emite $(chave, valor)$	Agrega (soma etc.) por chave

Custo computacional

O **custo de comunicação** costuma dominar o custo de processamento em si (ler do disco para memória é mais caro que a computação). Para um join $R(A, B) \bowtie S(B, C)$ com tamanhos r, s , o custo de comunicação é $O(r + s)$.

Exemplo — multiplicação de matrizes $n \times n$: com taxa de replicação $r = n$ e tamanho de redutor $q = 2n$, obtém-se $qr \geq 2n^2$, ou seja, limite inferior $\Omega(n^2)$ de comunicação. A teoria geral (Afrati & Ullman) formaliza o trade-off: **replicação $\times$ tamanho do redutor** — mais paralelismo (redutores menores) exige enviar cada dado para mais lugares. É um “princípio de conservação” análogo, em espírito, aos trade-offs de Shannon: não existe almoço grátis, apenas escolhas de onde pagar o custo.

13.3 Processamento em fluxo (Streaming) e Apache Spark

Enquanto o MapReduce clássico processa dados **em lote** (*batch*), o **Streaming Processing** é orientado a eventos: fluxo contínuo de pequenos pacotes, alto número de transações por segundo, em modo de transmissão imprevisível (“*burst mode*”), com execuções quase em tempo real (critério típico: 90%+ das operações concluídas em menos de 1 segundo).

O **Apache Spark** processa dados inteiramente **em memória** (ao contrário do Hadoop clássico, que grava resultados intermediários em disco — em jobs iterativos, como treinar um modelo de ML com dezenas de passadas sobre os dados, essa diferença chega a 10–100 $\times$ de desempenho), usando a abstração de **RDD** (*Resilient Distributed Dataset*), com suporte a streaming via DStream. A resiliência do RDD é uma ideia elegante: em vez de replicar os dados intermediários, o Spark guarda a *linhagem* (o grafo de transformações que os gerou — um DAG) e, se uma partição se perde, simplesmente a **recomputa** a partir da origem. Um **SparkContext** (dentro do *driver program*) coordena o cluster via um **Cluster Manager**, que distribui o trabalho entre *workers* executando *executores*.

O ecossistema em torno consolidou um vocabulário que vale conhecer: *Kafka* (filas de eventos distribuídas, a “artéria” dos pipelines de streaming), *Flink* (streaming nativo, evento a evento, em vez dos micro-batches do Spark) e, sobre o Spark, bibliotecas de alto nível (Spark SQL, MLlib, GraphX) que traduzem consultas declarativas para DAGs de RDDs.

13.4 Convergência com IA

Big Data e Inteligência Artificial são vistas como **técnicas convergentes**: IA se beneficia de plataformas de Big Data (PaaS, GPU clusters) para viabilizar treinamento em larga escala (pipelines de ETL — *Extract, Transform, Load*). Um exemplo prático: acelerar o Spark com GPUs via bibliotecas RAPIDS (NVIDIA).

Edge Computing e Federated Learning: em cenários de IoT, centralizar todo o dado para treinar um modelo gera latência e problemas de privacidade. O **Aprendizado Federado** resolve isso treinando modelos localmente em cada dispositivo de borda; apenas as **atualizações de parâmetros** (não os dados brutos) são enviadas a um servidor central, que agrega os modelos (ex.: algoritmo **FedAvg**, McMahan et al., 2017) — reduzindo tráfego de rede e preservando privacidade. Caso real: o teclado do Google (Gboard) aprende sugestões de próxima palavra federadamente, sem que o texto digitado saia do celular. Note a leitura informacional do truque: o gradiente médio de um modelo é um *resumo comprimido* (poucos MB) do que os dados locais

(GB) “têm a dizer” sobre a tarefa — transmite-se apenas a informação relevante, não o dado bruto.

Leituras complementares deste capítulo: DEAN & GHEMAWAT, “MapReduce: Simplified Data Processing on Large Clusters”, *OSDI*, 2004; LESKOVEC, RAJARAMAN & ULLMAN, *Mining of Massive Datasets*, 3^a ed. (caps. 2 e o material de custo de comunicação; gratuito em mmds.org); ZAHARIA et al., “Resilient Distributed Datasets”, *NSDI*, 2012; WHITE, T. *Hadoop: The Definitive Guide*, 4^a ed.; McMAHAN et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data”, *AISTATS*, 2017; KLEPPMANN, M. *Designing Data-Intensive Applications*, 2017 (visão sistêmica moderna, altamente recomendado).

Capítulo 14 — Fundamentos de Redes Neurais e Aprendizado

14.1 Por que aprendizado de máquina?

Sistemas especialistas (baseados em regras IF-THEN explícitas) funcionam bem quando as regras são claras, mas falham em tarefas perceptualmente simples para humanos, porém difíceis de codificar explicitamente (ex.: reconhecer o que há em uma imagem — tente escrever as regras IF-THEN que definem um gato...). A alternativa é imitar como humanos aprendem: **exposição a muitos dados + exemplos rotulados (labels)** → o próprio sistema identifica padrões.

Abordagem	Como funciona	Exige conhecimento de domínio?
Sistemas especialistas	Regras explícitas (IFs)	Sim, muito detalhado
Redes Neurais	Identifica <i>features</i> automaticamente	Exige dados bem estruturados
Deep Learning	Múltiplas camadas de <i>features</i> (hierárquicas)	Não exige estrutura fina, mas exige muito dado

Features (atributos de entrada): pixels (classificação de imagem), tom/volume (reconhecimento de voz), dados de câmeras/GPS (carros autônomos). Extrair *features* relevantes é essencial: uma *feature* irrelevante (ex.: hora do dia para classificar imagens de gatos) não ajuda; uma *feature* relevante em outro contexto (hora do dia para detectar SPAM, já que e-mails de SPAM costumam ser enviados de madrugada) pode ser crucial. Em linguagem do Capítulo 7: uma boa *feature* é aquela com **alta informação mútua com o alvo** — a hierarquia do *deep learning* pode ser lida como uma cadeia que destila, camada a camada, a informação da entrada que é relevante para o rótulo, descartando o resto (essa leitura é elaborada pela teoria do *Information Bottleneck*, Tishby et al.).

14.2 Métricas de avaliação

$$\text{Precisão} = \frac{VP}{VP + FP}, \quad \text{Recall} = \frac{VP}{VP + FN}, \quad \text{Acurácia} = \frac{VP + VN}{\text{total}}$$

(VP = verdadeiro positivo, FP = falso positivo, FN = falso negativo, VN = verdadeiro negativo)

Exemplo resolvido (por que acurácia engana): detector de fraude num conjunto com 1% de fraudes. O classificador preguiçoso “nunca é fraude” acerta 99% — acurácia altíssima, utilidade nula (recall = 0). Suponha um detector real que, em 10.000 transações (100 fraudes), sinaliza 150, das quais 80 são fraudes de fato: Precisão = $80/150 \approx 53\%$ (dos alarmes, quantos são reais); Recall = $80/100 = 80\%$ (das fraudes, quantas foram pegas); Acurácia = $(80 + 9830)/10000 = 99,1\%$ — quase a mesma do classificador inútil! Em classes desbalanceadas, olhe precisão/recall (ou sua média harmônica, o $F1\text{-score} = 2 \cdot \frac{P \cdot R}{P + R} \approx 0,64$ aqui), nunca só a acurácia. O trade-off precisão $\times$ recall é controlado pelo limiar de decisão — exatamente a matriz de perda da teoria da decisão do Capítulo 5.

14.3 O mecanismo de treinamento

Erro e função de perda

$$MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2, \quad RMSE = \sqrt{MSE}$$

O MSE é o segundo momento do erro (combina viés e variância do estimador) e é sensível a *outliers*. Treinar um modelo é, essencialmente, **minimizar uma função de perda** (*loss*) — no caso mais simples, o próprio MSE. (Lembre das conexões do Capítulo 5: MSE = MLE sob ruído gaussiano; e do Capítulo 7: *cross-entropy loss* = minimizar a KL até a distribuição dos rótulos. As perdas usuais não são arbitrárias — são estatística e teoria da informação disfarçadas.)

Gradiente descendente

$$w \leftarrow w - \alpha \cdot \frac{\partial L(w)}{\partial w}$$

onde α é a **taxa de aprendizagem** (*learning rate*). Um α muito alto causa divergência/oscilação; um α muito baixo torna a convergência lenta. Vocabulário associado: **época** (uma passada completa pelo dataset), **batch** (subconjunto de amostras usado por atualização), **step** (uma atualização de pesos).

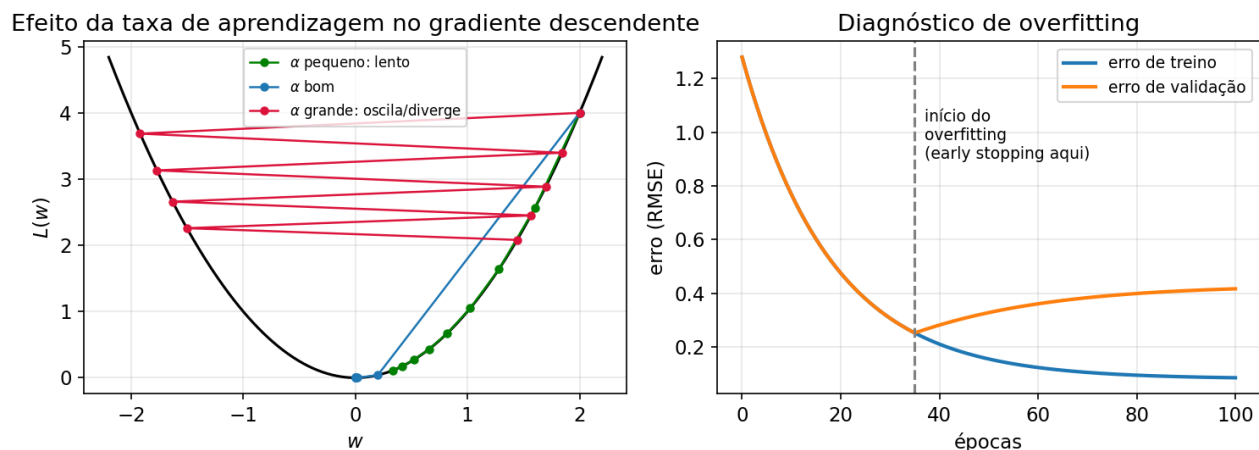


Figura 12: À esquerda, o efeito da taxa de aprendizagem sobre a trajetória do gradiente descendente em $L(w) = w^2$. À direita, as curvas de treino × validação e o ponto de early stopping.

Exemplo resolvido (uma iteração, à mão): modelo $\hat{y} = wx$, dado único $(x, y) = (2, 8)$, $w_0 = 1$, $\alpha = 0,05$, perda $L = (y - wx)^2$. Gradiente: $\frac{\partial L}{\partial w} = -2x(y - wx) = -2 \cdot 2 \cdot (8 - 2) = -24$. Atualização: $w_1 = 1 - 0,05 \cdot (-24) = 2,2$. Nova perda: $(8 - 4,4)^2 = 12,96$ (antes: 36). Repetindo, w converge ao ótimo $w^* = 4$. Com $\alpha = 0,3$, a primeira atualização saltaria para $w_1 = 8,2$ — do outro lado do vale, mais longe do ótimo: divergência.

Variantes do Gradiente Descendente

Variante	Fórmula	Características
Vanilla (Batch) GD	$\theta = \theta - \eta \nabla_{\theta} J(\theta)$ (dataset inteiro)	Lento, garante convergência ao mínimo global (em funções convexas)
SGD (estocástico)	$\theta = \theta - \eta \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)})$ (1 exemplo)	Rápido, mas oscila em torno do mínimo
Mini-batch GD	Sobre um lote de n amostras (tipicamente 32-64)	Equilíbrio entre velocidade e estabilidade — o mais usado na prática

Para acelerar a convergência em superfícies de perda “estreitas” (ravinas), usa-se **Momentum**: $v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta)$, que acumula uma fração γ do gradiente anterior (a analogia física: uma bola pesada descendo o vale atravessa pequenas ondulações sem parar). Variantes mais avançadas (**Adam**, Adagrad, RMSprop) adaptam a taxa de aprendizagem por parâmetro — o Adam (Kingma & Ba, 2015) combina momentum com normalização pela variância dos gradientes e é o default de fato do deep learning atual.

Funções de ativação (a não-linearidade)

Sem uma função de ativação não-linear, uma rede de múltiplas camadas equivaleria a uma única transformação linear (**composição de funções lineares é linear**) — é a não-linearidade que dá às redes neurais seu poder expressivo.

$$\text{ReLU}(x) = \max(0, x), \quad \text{Sigmoid}(x) = \frac{1}{1 + e^{-x}}, \quad \text{Softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

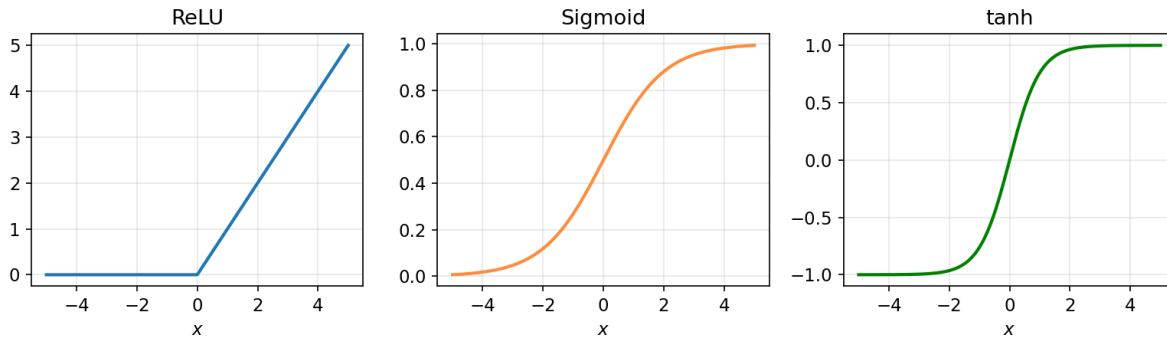


Figura 13: As três não-linearidades mais comuns. A ReLU dominou o deep learning por ter gradiente constante na região ativa (evita o “gradiente que desvanece” da sigmoid saturada) e custo computacional trivial.

Papéis típicos: ReLU nas camadas ocultas; sigmoid na saída de classificação binária (produz uma probabilidade); softmax na saída multiclasse (produz uma distribuição de probabilidade sobre as K classes — que alimenta diretamente a *cross-entropy* do Capítulo 7).

Backpropagation (regra da cadeia)

Uma rede é uma composição de módulos $X_i = F_i(X_{i-1}, W_i)$. Treinar significa calcular $\partial E / \partial W_i$ para todas as camadas, usando a **regra da cadeia**:

$$\frac{\partial E}{\partial W_i} = \frac{\partial E}{\partial X_i} \cdot \frac{\partial F_i(X_{i-1}, W_i)}{\partial W_i}$$

O algoritmo de **retropropagação** (*backpropagation*, popularizado por Rumelhart, Hinton & Williams, 1986) calcula isso eficientemente varrendo a rede **da saída para a entrada**: define-se $\delta_n = \partial C / \partial X_n$ na última camada, e propaga-se recursivamente $\delta_{i-1} = \delta_i \cdot \partial F_i / \partial X_{i-1}$, obtendo $\partial E / \partial W_i = \delta_i \cdot \partial F_i / \partial W_i$ em cada camada. O ponto genial é a **reutilização**: cada δ_i é computado uma única vez e serve a todos os pesos da camada — o custo de obter *todos* os gradientes é apenas $\sim 2 \times$ o custo de uma passada direta, em vez de um custo por parâmetro (que seria proibitivo em redes com bilhões de pesos).

14.4 Overfitting e boas práticas

Overfitting: quando um modelo se ajusta excessivamente aos dados de treino, capturando **ruído** em vez de padrão real, e por isso não generaliza bem para dados novos — detectável comparando o erro (RMSE) em treino vs. validação (ver figura da seção anterior: o erro de treino segue caindo enquanto o de validação volta a subir; o ponto da inflexão é o momento de parar — *early stopping*).

A leitura informacional do fenômeno fecha o ciclo com o Capítulo 12: um modelo com capacidade demais consegue “memorizar” os dados (descrição longa), em vez de encontrar a *regularidade curta* que os explica — overfitting é a violação prática da Navalha de Occam, e o princípio MDL o formaliza: minimize [bits do modelo] + [bits dos dados dado o modelo]. A regularização Ridge/Lasso (Capítulo 5) e o *dropout* são maneiras de forçar descrições mais curtas.

Conjuntos de dados:

Conjunto	Papel	Proporção típica
Treinamento	Ajusta os pesos	~80%
Validação	Avalia sem viés durante o ajuste (<i>cross-validation</i>)	~20%
Teste	Avaliação final, usada uma única vez	Conjunto separado e curado

Por que o teste só pode ser usado uma vez? Porque cada decisão tomada olhando para ele (escolher arquitetura, ajustar hiperparâmetro) *vaza informação* do teste para o modelo — e a estimativa de generalização deixa de ser honesta. Vazamentos mais sutis (normalizar usando estatísticas do conjunto inteiro antes de separar treino/teste, por exemplo) são erros clássicos de iniciantes: as estatísticas de normalização do Capítulo 3 devem ser calculadas **só no treino** e então aplicadas ao teste.

Batch Normalization: normalizar (Capítulo 3) apenas uma vez, antes do treino, “se perde” ao longo das atualizações de peso. Normalizar **dentro de cada batch, durante o treino** (Ioffe & Szegedy, 2015), resolve esse problema e costuma reduzir o número de épocas necessárias para convergir.

Leituras complementares deste capítulo: GOODFELLOW, BENGIO & COURVILLE, *Deep Learning*, MIT Press, 2016 (gratuito em deeplearningbook.org; caps. 6–8); NIELSEN, M. *Neural Networks and Deep Learning* (livro-web gratuito, a melhor introdução visual a backpropagation); RUMELHART, HINTON & WILLIAMS, “Learning representations by back-propagating errors”, *Nature*, 1986; KINGMA & BA, “Adam: A Method for Stochastic Optimization”, *ICLR*, 2015; IOFFE & SZEGEDY, “Batch Normalization”, *ICML*, 2015; TISHBY & ZASLAVSKY, “Deep Learning and the Information Bottleneck Principle”, 2015; LECUN, BENGIO & HINTON, “Deep Learning”, *Nature*, 2015 (survey).

Capítulo 15 — Análise de Componentes Principais (PCA)

15.1 Contexto histórico

O PCA (*Principal Component Analysis*) foi desenvolvido por Harold Hotelling em 1933 (também chamado de **Transformada de Hotelling**; a formulação geométrica original é ainda anterior — Karl Pearson, 1901); em processamento de sinais é conhecido como **Transformada de Karhunen-Loève**. É um caso particular da **Decomposição em Valores Singulares (SVD)** quando a matriz decomposta é a matriz de covariância dos dados — que é sempre quadrada, simétrica e definida positiva.

15.2 O problema: redução de dimensionalidade preservando informação

Dado um conjunto de N vetores de atributos $x_k \in \mathbb{R}^p$ organizados em uma matriz $X \in \mathbb{R}^{p \times N}$, o objetivo é encontrar uma transformação linear $z_k = Qx_k$ que produza vetores $z_k \in \mathbb{R}^q$ com $q \leq p$, **preservando o máximo de informação relevante** contida em X .

Por que reduzir dimensões, afinal? Três motivos práticos: (i) a **maldição da dimensionalidade** — em alta dimensão, os dados ficam esparsos e as distâncias perdem contraste, degradando KNN e clustering (Capítulo 5); (ii) **visualização** — humanos enxergam até 3D; (iii) **atributos reais são redundantes** — altura e envergadura de braços, temperatura de sensores vizinhos: fortemente correlacionados, quase a mesma informação repetida. O PCA remove exatamente essa redundância *linear*. Note a escolha implícita: aqui, “informação” é operacionalizada como **variância** — uma medida linear e de segunda ordem, mais simples que a entropia (a versão “entrópica” do problema é a ICA, mencionada no Capítulo 9).

15.3 O algoritmo PCA, passo a passo

Passo 1 — Vetor-média: $\bar{x} = \frac{1}{N} \sum_{k=1}^N x_k$

Passo 2 — Centralização: $x_k \leftarrow x_k - \bar{x}$ (daqui em diante, x_k já está centralizado)

Passo 3 — Matriz de covariância: $C_x = E[xx^T] \approx \frac{1}{N} \sum_{k=1}^N x_k x_k^T$

Passo 4 — Autovalores e autovetores: resolver o problema de autovalor $C_x v = \lambda v$, cujas soluções não-triviais exigem

$$\det(C_x - \lambda I_p) = 0$$

Como C_x é **simétrica e definida positiva**, todos os p autovalores são **reais e positivos**. Convencionou-se ordená-los de forma decrescente: $\lambda_1 > \lambda_2 > \dots > \lambda_p$.

Exemplo numérico resolvido ($p = 2$):

$$C_x = \begin{bmatrix} 1 & 0,8 \\ 0,8 & 4 \end{bmatrix} \Rightarrow \det(C_x - \lambda I) = \lambda^2 - 5\lambda + 3,36 = 0 \Rightarrow \lambda_1 = 4,2, \lambda_2 = 0,8$$

Nota de correção: alguns slides do curso invertem a ordem ($\lambda_1 = 0,8$, $\lambda_2 = 4,2$) — mas por convenção, λ_1 é sempre o **maior** autovalor.

Completando o exemplo — os autovetores. Para $\lambda_1 = 4,2$: $(C_x - 4,2I)v = 0$ dá $-3,2v_x + 0,8v_y = 0$, ou seja, $v_y = 4v_x$. Normalizando: $v_1 = \frac{1}{\sqrt{17}}(1, 4)^T \approx (0,243; 0,970)^T$. Para $\lambda_2 = 0,8$: $v_2 = \frac{1}{\sqrt{17}}(4, -1)^T \approx (0,970; -0,243)^T$. Verifique: $v_1^T v_2 = 0$ (ortogonais) e cada um tem norma 1. A primeira componente aponta “quase na vertical” — coerente com a covariância: a segunda variável tem variância 4 (contra 1 da primeira) e domina a direção de máxima dispersão.

Transformando um ponto: para $x = (1, 2)^T$ (já centralizado), $z = V^T x = (v_1^T x, v_2^T x) = \left(\frac{1+8}{\sqrt{17}}, \frac{4-2}{\sqrt{17}}\right) \approx (2,18; 0,49)$ — as novas coordenadas do mesmo ponto, na base dos autovetores.

Os autovetores v_i formam um conjunto **ortonormal** ($v_i^T v_j = 1$ se $i = j$, 0 caso contrário), dispostos em colunas na matriz $V = [v_1 \ v_2 \ \dots \ v_p]$. Como $VV^T = I_p$, tem-se $V^{-1} = V^T$ — propriedade essencial para reconstruir os dados originais a partir dos transformados.

Observação sobre não-unicidade: se v_i é solução, $-v_i$ também é (basta verificar: $C_x(-v_i) = \lambda_i(-v_i)$) — diferentes implementações numéricas (ex.: eig vs. pcacov no Octave/MATLAB) podem retornar autovetores com sinais diferentes, sem que isso afete a análise.

Passo 5 — Matriz de transformação: $Q = V^T$

Passo 6 — Transformação: $z_k = Qx_k$ (ou, em forma matricial, $Z = QX$). O mapeamento inverso é $x_k = Q^T z_k$ (ou $X = Q^T Z$).

15.4 Por que isso funciona? A diagonalização da covariância

O resultado central do PCA é entender o que acontece à covariância dos dados após a transformação:

$$C_z = E[zz^T] = E[(V^T x)(V^T x)^T] = V^T E[xx^T]V = V^T C_x V$$

Usando $C_x v_i = \lambda_i v_i$ e a ortonormalidade dos v_i :

$$C_z = V^T C_x V = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_p)$$

Conclusões-chave:

1. A covariância dos dados transformados é **diagonal** — as componentes de z são **descorrelacionadas** entre si.
2. As variâncias das novas variáveis z_i são **exatamente os autovalores** de C_x : $\lambda_i = \sigma_i^2$. Não é preciso recalcular nada — os autovalores já foram obtidos no Passo 4.

No exemplo numérico: a variância total original era $1 + 4 = 5$; após o PCA, $4,2 + 0,8 = 5$ — o traço da matriz se conserva (a rotação não cria nem destrói variância, só a **reorganiza**: 84% dela agora mora na primeira componente).

15.5 Interpretação geométrica: mudança de base

Uma **base** de $\mathbb{R}^p$ é qualquer conjunto de p vetores linearmente independentes; uma base é **ortonormal** se seus vetores são mutuamente ortogonais e de norma unitária. Os dados originais x_k estão representados na base canônica; os dados transformados z_k estão representados na base formada pelos **autovetores da matriz de covariância**.

Geometricamente, o PCA corresponde a uma **rotação** do sistema de coordenadas: os novos eixos ($s_1, s_2, \dots$) se alinham com as **direções de maior variância** dos dados. No espaço original, os atributos costumam estar correlacionados (nuvem de pontos “inclinada”); depois do PCA, a nuvem fica alinhada aos eixos (descorrelacionada) — o primeiro eixo captura a direção de máxima dispersão, o segundo a maior dispersão ortogonal ao primeiro, e assim sucessivamente.

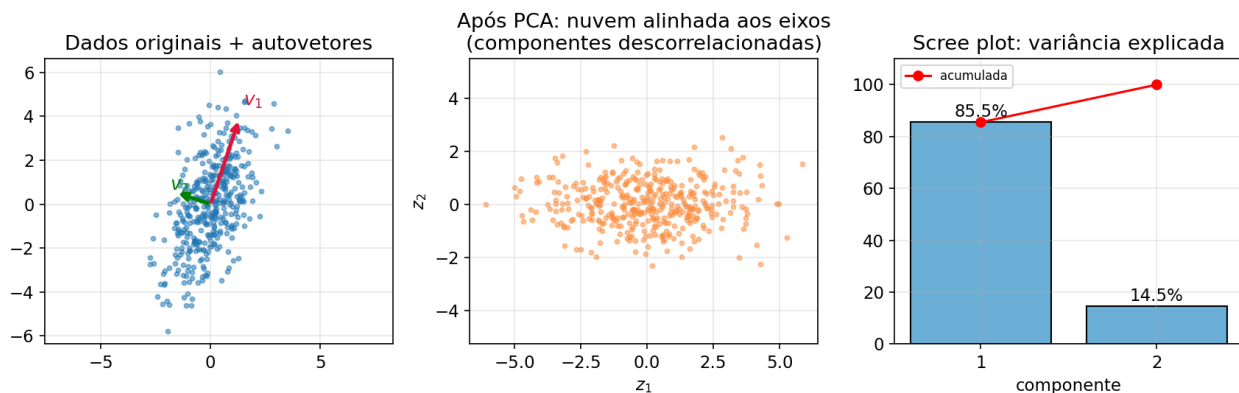


Figura 14: PCA em ação sobre 400 pontos amostrados com a covariância do exemplo numérico. Esquerda: dados originais com os autovetores (setas, escaladas por $\sqrt{\lambda_i}$). Centro: os mesmos dados na base dos autovetores. Direita: o scree plot correspondente.

15.6 Aplicações

Seleção de atributos

A primeira componente $z_1 = v_1^T x = v_{11}x_1 + v_{21}x_2 + \dots + v_{p1}x_p$ é a projeção dos dados na direção de maior variância. Quanto maior o coeficiente $|v_{j1}|$, mais importante é a variável original x_j para explicar a variância principal dos dados — usado como critério de **seleção de atributos**.

Redução de dimensionalidade

Usando apenas as q primeiras colunas de V (associadas aos q maiores autovalores), forma-se $V_q (p \times q)$ e $Q_q = V_q^T$. A transformação $z_k = Q_q x_k$ produz vetores de dimensão $q < p$ — uma **compressão com perdas**, análoga em espírito à compressão de dados do Capítulo 9, mas para dados numéricos multivariados em vez de símbolos discretos.

Variância total e variância explicada

$$VT = \sum_{i=1}^p \lambda_i \quad (\text{variância total, medida linear de "informação"})$$

$$VE_i = 100 \times \frac{\lambda_i}{VT} \quad (\text{variância explicada pela componente } i), \quad VE(q) = 100 \times \frac{\sum_{i=1}^q \lambda_i}{VT} \quad (\text{acumulada})$$

O gráfico de $VE(q)$ em função de q é o **scree plot**, usado para decidir quantas componentes reter (tipicamente onde a curva “dobra” — o “cotovelo”; critérios alternativos: reter componentes até acumular 90-95% da variância, ou reter apenas λ_i acima da média — *critério de Kaiser*). Se $q = p$, não há perda de informação (reconstrução perfeita); se $q < p$, há perda proporcional à variância descartada — um trade-off explícito entre compressão e fidelidade, no mesmo espírito da compressão com perdas do Capítulo 9. No exemplo numérico, reter só a primeira componente comprime os dados em 50% (de 2 números para 1 por ponto) preservando 84% da variância.

Duas aplicações célebres: (i) **Eigenfaces** (Turk & Pentland, 1991) — rostos 100×100 vivem em $\mathbb{R}^{10.000}$, mas ~ 50 -150 componentes principais (“autofaces”) bastam para reconhecê-los: cada rosto vira um vetor de ~ 100 coeficientes; (ii) **pré-processamento/decorrelação** — comprimir a dimensão de entrada antes de treinar modelos (Capítulo 14) reduz custo e ruído. **Advertências de uso:** o PCA é sensível a escala — atributos em unidades díspares devem ser padronizados antes (z-score, Capítulo 3), caso contrário o atributo de maior variância numérica “sequestra” a primeira componente; e ele só enxerga estrutura *linear* — dados em espiral ou variedades curvas pedem métodos não-lineares (kernel PCA, t-SNE, UMAP, autoencoders).

Leituras complementares deste capítulo: HOTELLING, “Analysis of a Complex of Statistical Variables into Principal Components”, 1933;

PEARSON, “On Lines and Planes of Closest Fit to Systems of Points in Space”, 1901; JOLLIFFE, I. *Principal Component Analysis*, 2^a ed., Springer (a monografia da área); SHLENS, J. “A Tutorial on Principal Component Analysis”, arXiv:1404.1100 (o tutorial curto mais recomendado); TURK & PENTLAND, “Eigenfaces for Recognition”, *J. Cognitive Neuroscience*, 1991; STRANG, G. *Linear Algebra and Learning from Data*, 2019 (para o pano de fundo de autovalores/SVD).

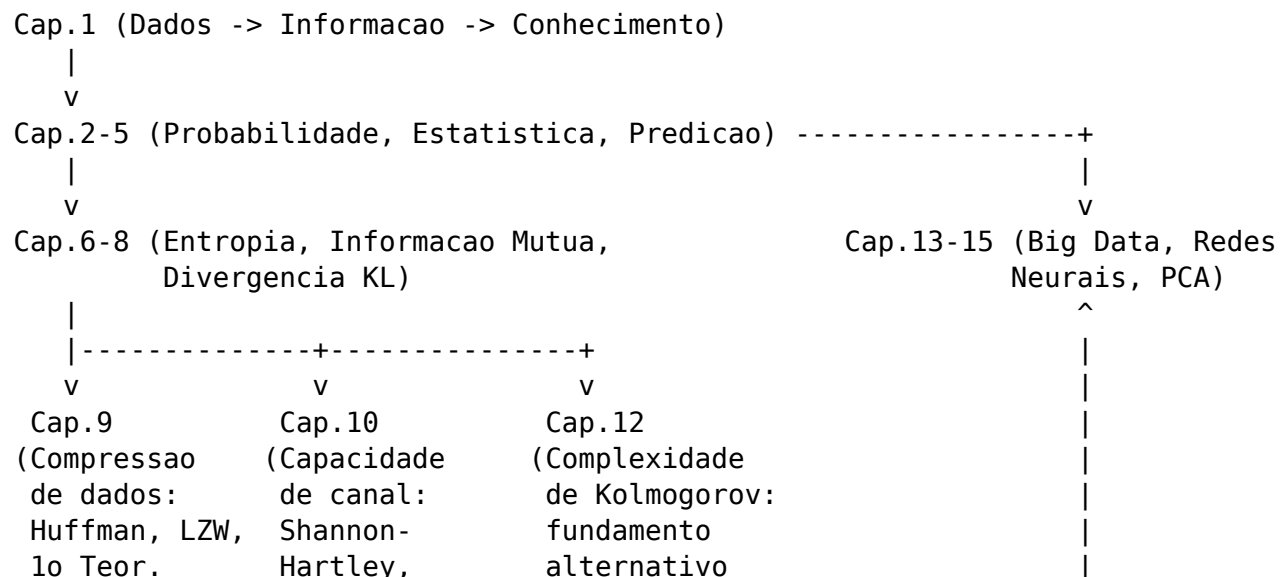
Capítulo 16 — Síntese Final e Mapa Conceitual

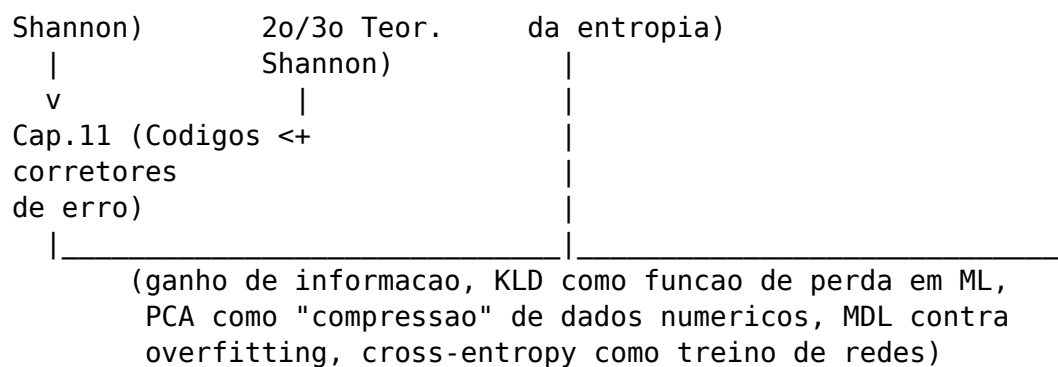
16.1 O fio condutor do curso

Se há uma ideia que amarra todos os quinze capítulos anteriores, é esta: **informação pode ser quantificada matematicamente, e essa quantificação tem consequências práticas precisas** — quantos bits, em média, um símbolo carrega (entropia); quantos bits, no mínimo, uma mensagem pode ser comprimida (Primeiro Teorema de Shannon); quantos bits por segundo um canal ruidoso consegue transportar de forma confiável (Shannon-Hartley); quantos bits, no mínimo, descrevem um objeto (Complexidade de Kolmogorov); e quantas dimensões, no mínimo, capturam a maior parte da variância de um conjunto de dados (PCA).

Dito de outra forma: o curso apresenta **três limites fundamentais** (entropia para compressão, capacidade para transmissão, complexidade para descrição) e **um arsenal de técnicas** que tentam se aproximar deles (Huffman/LZW; códigos corretores; modelos estatísticos e de aprendizado). A engenharia inteira da era digital vive no vão entre os limites e as técnicas.

16.2 Mapa de conexões entre os capítulos





16.3 Tabela-resumo das grandezas fundamentais

Grandeza	Fórmula essencial	O que responde
Autoinformação $I(a_k)$	$-\log_2 p_k$	Quanta surpresa carrega um evento?
Entropia $H(A)$	$-\sum_k p_k \log_2 p_k$	Quantos bits, em média, por símbolo?
Entropia condicional $H(A B)$	$H(A, B) - H(B)$	Quanta incerteza sobre A , sabendo B ?
Informação mútua $I(A, B)$	$H(A) - H(A B)$	Quanto B me diz sobre A ?
Divergência KL $D(p\ g)$	$\sum p \log(p/g)$	Quão diferente é p de g ? (custo de modelar errado)
Comprimento médio ótimo L_{min}	$= H(A)$	Menor taxa média de compressão sem perdas
Capacidade de canal C	$B \log_2(1 + SNR)$	Máxima taxa confiável de transmissão
Complexidade de Kolmogorov $K(x)$	$\min\{l(p) : U(p) = x\}$	Menor programa que gera x
Variância explicada $VE(q)$	$\sum_{i \leq q} \lambda_i / \sum_i \lambda_i$	Quanta informação (linear) as q primeiras componentes retêm?

16.4 Dicas de estudo e revisão

- **Para quem está vendo pela primeira vez:** siga a ordem dos capítulos. A Parte I (probabilidade e estatística) é pré-requisito indispensável para a Parte II; a Parte II, por sua vez, fundamenta boa parte do vocabulário usado na Parte III (ganho de informação em árvores de decisão, divergência KL como função de perda, entropia como critério de otimização).
- **Para quem está revisando:** a Tabela-resumo (16.3) e o Mapa de Conexões (16.2) servem como índice rápido; cada capítulo tem exemplos numéricos resolvidos que podem ser refeitos como exercício de fixação (Bayes diagnóstico e cadeia de Markov no Cap. 5, dado viciado no Cap. 6, canal conjunto no Cap. 7, Huffman

e LZW no Cap. 9, Shannon-Hartley e o modem no Cap. 10, Hamming(7,4) no Cap. 11, PCA 2×2 completo no Cap. 15, gradiente à mão no Cap. 14).

- **Roteiro de auto-teste sugerido (uma pergunta por parte):** (I) *“Por que covariância nula não implica independência, e qual grandeza do curso detecta a dependência que a correlação não vê?”* (II) *“Deduza a capacidade do BSC a partir de $C = \max I(X, Y)$ e explique por que $C(0,5) = 0$.”* (III) *“Explique overfitting em linguagem de MDL/Kolmogorov.”* Se as três respostas saírem fluentes, a espinha dorsal do curso está firme.
- **Erros e imprecisões dos materiais originais:** ao longo da consolidação deste texto, alguns pontos dos slides originais foram corrigidos ou esclarecidos (ex.: direção da desigualdade de Jensen, ordenação de autovalores no exemplo de PCA, a equação 58 de entropia condicional diferencial). Essas correções estão sinalizadas em caixas de nota ao longo do texto.

Referências e leituras complementares

Materiais originais da disciplina (Prof. Dr. Julio César Santos dos Anjos):

- Módulos 01-06 de Tecnologia da Informação (slides).
- Métricas — Revisão — Pré-Processamento de Dados.
- Apostila UFC — Introdução à Teoria da Informação e Funções Estatísticas Importantes.
- Análises de Componentes Principais — PCA — Principais Conceitos.
- Complexidade de Kolmogorov (capítulo de Bruno Durand e Alexander Zvonkin).

Bibliografia citada nos materiais e recomendada para aprofundamento:

- SHANNON, C. E. *A Mathematical Theory of Communication*. Bell System Technical Journal, 1948.
- COVER, T. M.; THOMAS, J. A. *Elements of Information Theory*. 2ª ed., Wiley (referência clássica para aprofundar entropia diferencial, capacidade de canal e os teoremas de Shannon em forma matemática precisa).
- BISHOP, C. M. *Pattern Recognition and Machine Learning*. Springer, 2009.
- HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning*. 2ª ed., 2011.
- ROSS, S. *Probabilidade: Um Curso Moderno com Aplicações*. 2010.
- LESKOVEC, J.; RAJARAMAN, A.; ULLMAN, J. D. *Mining of Massive Datasets*. 3ª ed., Cambridge University Press, 2020.
- DEAN, J.; GHEMAWAT, S. *MapReduce: Simplified Data Processing on Large Clusters*. OSDI, 2004.
- KOLMOGOROV, A. N. *Three Approaches to the Definition of the Concept of "Quantity of Information"*. 1965.
- HOFSTADTER, D. *Gödel, Escher, Bach: An Eternal Golden Braid*. 1979.
- HOTELLING, H. *Analysis of a Complex of Statistical Variables into Principal Components*. Journal of Educational Psychology, 1933.
- RÉNYI, A. *On Measures of Information and Entropy*. Proceedings of the 4th Berkeley Symposium, 1960.

Novas fontes incorporadas nesta edição (também listadas ao fim de cada capítulo):

- MACKAY, D. J. C. *Information Theory, Inference, and Learning Algorithms*. Cambridge, 2003 — gratuito em inference.org.uk/mackay/itila; provavelmente o melhor segundo livro do curso inteiro, por unir Shannon, códigos e aprendizado

- bayesiano num só volume, com exercícios excepcionais.
- SHANNON, C. E. *Prediction and Entropy of Printed English*. Bell System Technical Journal, 1951.
 - BLITZSTEIN, J.; HWANG, J. *Introduction to Probability*. 2ª ed., CRC, 2019 (com curso gratuito “Stat 110”).
 - BUSSAB, W.; MORETTIN, P. *Estatística Básica*. 9ª ed., Saraiva (referência em português para a Parte I).
 - CASELLA, G.; BERGER, R. *Statistical Inference*. 2ª ed., Duxbury.
 - TUKEY, J. *Exploratory Data Analysis*. Addison-Wesley, 1977.
 - JAYNES, E. T. *Information Theory and Statistical Mechanics*. Physical Review, 1957 (princípio da máxima entropia).
 - HUFFMAN, D. *A Method for the Construction of Minimum-Redundancy Codes*. Proc. IRE, 1952.
 - ZIV, J.; LEMPEL, A. *A Universal Algorithm for Sequential Data Compression*. IEEE Trans. Information Theory, 1977; WELCH, T., IEEE Computer, 1984.
 - SAYOOD, K. *Introduction to Data Compression*. 5ª ed., Morgan Kaufmann.
 - PRINCIPE, J. C. *Information Theoretic Learning: Rényi’s Entropy and Kernel Perspectives*. Springer, 2010.
 - HAYKIN, S. *Sistemas de Comunicação*. 5ª ed., Bookman (em português).
 - SKLAR, B. *Digital Communications: Fundamentals and Applications*. 2ª ed., Prentice Hall.
 - LIN, S.; COSTELLO, D. *Error Control Coding*. 2ª ed., Prentice Hall.
 - BERROU, C.; GLAVIEUX, A.; THITIMAJSHIMA, P. *Near Shannon Limit Error-Correcting Coding: Turbo-Codes*. ICC, 1993; GALLAGER, R. *Low-Density Parity-Check Codes*, MIT Press, 1963; ARIKAN, E., IEEE Trans. IT, 2009 (códigos polares).
 - LI, M.; VITÁNYI, P. *An Introduction to Kolmogorov Complexity and Its Applications*. 4ª ed., Springer, 2019.
 - GRÜNWALD, P. *The Minimum Description Length Principle*. MIT Press, 2007.
 - CILIBRASI, R.; VITÁNYI, P. *Clustering by Compression*. IEEE Trans. Information Theory, 2005.
 - ZAHARIA, M. et al. *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*. NSDI, 2012.
 - KLEPPMANN, M. *Designing Data-Intensive Applications*. O’Reilly, 2017.
 - McMAHAN, H. B. et al. *Communication-Efficient Learning of Deep Networks from Decentralized Data*. AISTATS, 2017 (FedAvg).
 - GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016 — gratuito em deeplearningbook.org.
 - NIELSEN, M. *Neural Networks and Deep Learning*. Livro-web gratuito, neuralnetworksanddeeplearning.com.
 - LECUN, Y.; BENGIO, Y.; HINTON, G. *Deep Learning*. Nature, 2015.
 - JOLLIFFE, I. *Principal Component Analysis*. 2ª ed., Springer, 2002.
 - SHLENS, J. *A Tutorial on Principal Component Analysis*. arXiv:1404.1100, 2014.
 - TURK, M.; PENTLAND, A. *Eigenfaces for Recognition*. Journal of Cognitive Neuroscience, 1991.
 - STRANG, G. *Linear Algebra and Learning from Data*. Wellesley-Cambridge, 2019.
 - GLEICK, J. *A Informação: uma história, uma teoria, uma enxurrada*. Companhia das Letras, 2013 (edição brasileira) — leitura de divulgação para dar contexto

humano a todo o curso.

Apostila consolidada a partir dos materiais de curso e organizada — e, nesta edição, ampliada com exemplos, figuras e leituras adicionais — para uso como referência única de estudo e revisão da disciplina de Teoria da Informação.